

DATA-260 Homework 4 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16" (Ryzen 5 7640HS, 16GB RAM, NVIDIA RTX 4060 8GB, 512GB SSD)

Local model used: qwen3:8b via Ollama, temperature 0.0 (Part 4 generation); sentence-transformers/all-MiniLM-L6-v2 (Part 4 embeddings); FastAPI + MySQL 8 via SQLAlchemy (Parts 1–3); React + Vite (Part 1)

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: PLACEHOLDER_HASH (to be filled in after the final commit and hw4 tag)
<https://github.com/UdayPate/data260-9871/tree/f25b97d4b9d959d750e812f2be3651c80ac67eda>

Collaborator access: confirmed working for Sbnikitha and supriyaselvangesan.

Part 1 – React Client

The existing FastAPI backend from HW1–HW3 was extended (not replaced) with a JSON API (`api_v2.py`) that a new React client talks to, alongside the original server-rendered pages, which are left in place. The client is a Vite + React project at `code/web_application/client`, using `react-router-dom` for routing and the `useState/useEffect` hooks throughout. `App.jsx` owns the fixtures list and the create/update/delete functions, and passes them down as props to the Create, Update and Delete components, per the assignment's requirement.

On first load, `App.jsx` calls `GET /api/auth/me` to ask the backend "am I logged in?" – this is what lets a page refresh keep the user logged in, since React itself has no memory of anything; the HTTP-only session cookie is what actually persists. If not logged in, the fixture list and record actions are hidden and a `Login required` message is shown instead. Note: `UpdateRecord.jsx` and `DeleteRecord.jsx` render at `/update/:id` and `/delete/:id` rather than a bare `/update` or `/delete` – a route with no identifier cannot know which of many records to act on, so an `id` parameter was added; the routes otherwise render the exact components the assignment describes.

App.jsx – routing, auth check, and the CRUD functions passed down as props

```
import { useState, useEffect } from "react";
import { BrowserRouter, Routes, Route, Link, Navigate } from "react-router-dom";
import { api } from "../api";
import Login from "../Login";
import Home from "../Home";
import CreateRecord from "../CreateRecord";
import UpdateRecord from "../UpdateRecord";
import DeleteRecord from "../DeleteRecord";

export default function App() {
  const [user, setUser] = useState(null);
  const [authChecked, setAuthChecked] = useState(false);
  const [fixtures, setFixtures] = useState([]);

  useEffect(() => {
    api
      .me()
      .then((u) => setUser(u))
      .catch(() => setUser(null))
      .finally(() => setAuthChecked(true));
  }, []);

  useEffect(() => {
    if (user) {
      api.listFixtures().then(setFixtures).catch(() => setFixtures([]));
    } else {
      setFixtures([]);
    }
  }, [user]);

  async function handleLogout() {
    await api.logout();
    setUser(null);
  }

  async function addFixture(data) {
    const created = await api.createFixture(data);
    setFixtures((prev) => [...prev, created]);
  }

  async function editFixture(id, data) {
    const updated = await api.updateFixture(id, data);
    setFixtures((prev) => prev.map((f) => (f.id === Number(id) ? updated : f)));
  }

  async function removeFixture(id) {
    await api.deleteFixture(id);
    setFixtures((prev) => prev.filter((f) => f.id !== Number(id)));
  }

  if (!authChecked) {
    return <p>Loading...</p>;
  }

  return (
    <BrowserRouter>
      <div className="app-shell">
        <nav className="app-nav">
```

```

    <Link to="/">Home</Link>
    {user && <Link to="/create">Add Record</Link>}}
    <span className="spacer"></span>
    {user ? (
      <>
        <span className="user-tag">Logged in as {user.name}</span>
        <button onClick={handleLogout}>Log out</button>
      </>
    ) : (
      <Link to="/login">Login</Link>
    )}
  </nav>

  <Routes>
    <Route path="/" element={<Home user={user} fixtures={fixtures} />} />
    <Route path="/login" element={<Login onLoginSuccess={setUser} />} />
    <Route
      path="/create"
      element={user ? <CreateRecord onCreate={addFixture} /> : <Navigate to="/login" />}
    />
    <Route
      path="/update/:id"
      element={user ? <UpdateRecord fixtures={fixtures} onUpdate={editFixture} /> : <Navigate to
="/login" />}
    />
    <Route
      path="/delete/:id"
      element={user ? <DeleteRecord fixtures={fixtures} onDelete={removeFixture} /> : <Navigate
to="/login" />}
    />
  </Routes>
</div>
</BrowserRouter>
);
}

```

I. Home Page (Home.jsx) – renders at /

```

import { Link } from "react-router-dom";

export default function Home({ user, fixtures }) {
  if (!user) {
    return <div className="page-card"><p>Login required to view fixtures.</p></div>;
  }

  return (
    <div className="page-card">
      <h2>Fixture Board</h2>
      {fixtures.length === 0 ? (
        <p>No fixtures yet.</p>
      ) : (
        <table className="fixture-table">
          <thead>
            <tr><th>ID</th><th>Fixture</th><th>Teams / Players</th><th>Actions</th></tr>
          </thead>
          <tbody>
            {fixtures.map((f) => (
              <tr key={f.id}>
                <td>{f.id}</td>

```

```

        <td>{f.fixture_name}</td>
        <td>{f.teams_players}</td>
        <td>
            <Link to={`/update/${f.id}`}>Edit</Link>
            <Link to={`/delete/${f.id}`}>Delete</Link>
        </td>
    </tr>
  </tbody>
</table>
  </div>
);
}

```

II. Add a New Record (CreateRecord.jsx) – renders at /create

Accepts `onCreate` as a prop (the function `App.jsx` passes down), takes the primary and secondary fields, and on submit calls the prop function then navigates home.

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";

export default function CreateRecord({ onCreate }) {
  const [fixtureName, setFixtureName] = useState("");
  const [teamsPlayers, setTeamsPlayers] = useState("");
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    await onCreate({ fixture_name: fixtureName, teams_players: teamsPlayers });
    navigate("/");
  }

  return (
    <div className="page-card">
      <h2>Add Fixture</h2>
      <form onSubmit={handleSubmit}>
        <div className="field">
          <label>Fixture Name</label>
          <input value={fixtureName} onChange={(e) => setFixtureName(e.target.value)} required />
        </div>
        <div className="field">
          <label>Teams / Players</label>
          <input value={teamsPlayers} onChange={(e) => setTeamsPlayers(e.target.value)} required />
        </div>
        <button className="btn-primary" type="submit">Add Fixture</button>
      </form>
    </div>
  );
}

```

III. Update Record (UpdateRecord.jsx) – renders at /update/:id

Accepts `fixtures` and `onUpdate` as props; `useEffect` pre-fills the form from the existing record when the id in the URL changes.

```

import { useState, useEffect } from "react";

```

```

import { useParams, useNavigate } from "react-router-dom";

export default function UpdateRecord({ fixtures, onUpdate }) {
  const { id } = useParams();
  const navigate = useNavigate();
  const [fixtureName, setFixtureName] = useState("");
  const [teamsPlayers, setTeamsPlayers] = useState("");

  useEffect(() => {
    const existing = fixtures.find((f) => f.id === Number(id));
    if (existing) {
      setFixtureName(existing.fixture_name);
      setTeamsPlayers(existing.teams_players);
    }
  }, [id, fixtures]);

  async function handleSubmit(e) {
    e.preventDefault();
    await onUpdate(id, { fixture_name: fixtureName, teams_players: teamsPlayers });
    navigate("/");
  }

  return (
    <div className="page-card">
      <h2>Update Fixture #{id}</h2>
      <form onSubmit={handleSubmit}>
        <div className="field">
          <label>Fixture Name</label>
          <input value={fixtureName} onChange={(e) => setFixtureName(e.target.value)} required />
        </div>
        <div className="field">
          <label>Teams / Players</label>
          <input value={teamsPlayers} onChange={(e) => setTeamsPlayers(e.target.value)} required />
        </div>
        <button className="btn-primary" type="submit">Save Changes</button>
      </form>
    </div>
  );
}

```

IV. Delete Record (DeleteRecord.jsx) – renders at /delete/:id

```

import { useParams, useNavigate } from "react-router-dom";

export default function DeleteRecord({ fixtures, onDelete }) {
  const { id } = useParams();
  const navigate = useNavigate();
  const fixture = fixtures.find((f) => f.id === Number(id));

  async function handleDelete() {
    await onDelete(id);
    navigate("/");
  }

  return (
    <div className="page-card">
      <h2>Delete Fixture</h2>
      {fixture ? (
        <p>Are you sure you want to delete "{fixture.fixture_name}"?</p>

```

```

    ) : (
      <p>Fixture #{id}</p>
    )}
    <button className="btn-primary" onClick={handleDelete}>Delete</button>
  </div>
);
}

```

Login.jsx – email + password login

```

import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { api } from "../api";

export default function Login({ onLoginSuccess }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    setError(null);
    try {
      const user = await api.login(email, password);
      onLoginSuccess(user);
      navigate("/");
    } catch (err) {
      setError("Invalid email or password.");
    }
  }

  return (
    <div className="page-card">
      <h2>League Admin Login</h2>
      {error && <p className="error-text">{error}</p>}
      <form onSubmit={handleSubmit}>
        <div className="field">
          <label>Email</label>
          <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required />
        </div>
        <div className="field">
          <label>Password</label>
          <input type="password" value={password} onChange={(e) => setPassword(e.target.value)}
            required />
        </div>
        <button className="btn-primary" type="submit">Log In</button>
      </form>
    </div>
  );
}

```

Screenshots

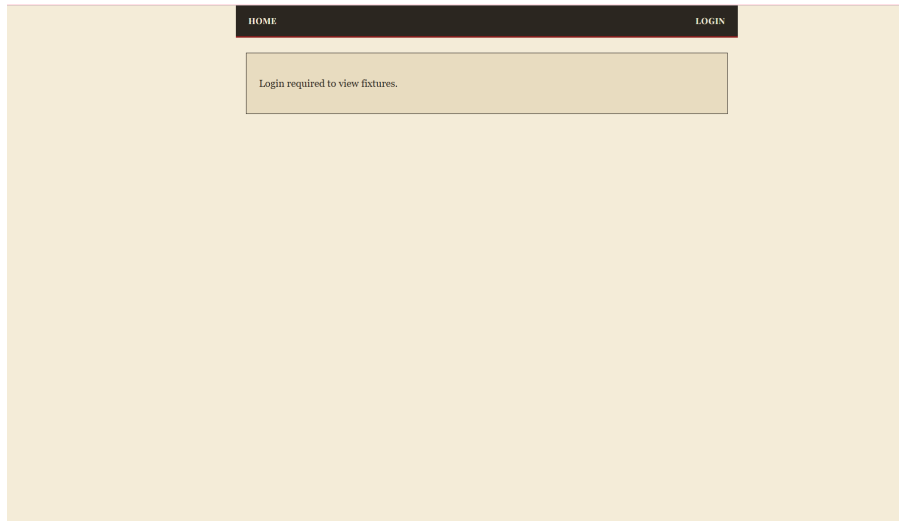


Figure 1: Home page (/), logged out – shows "Login required to view fixtures."

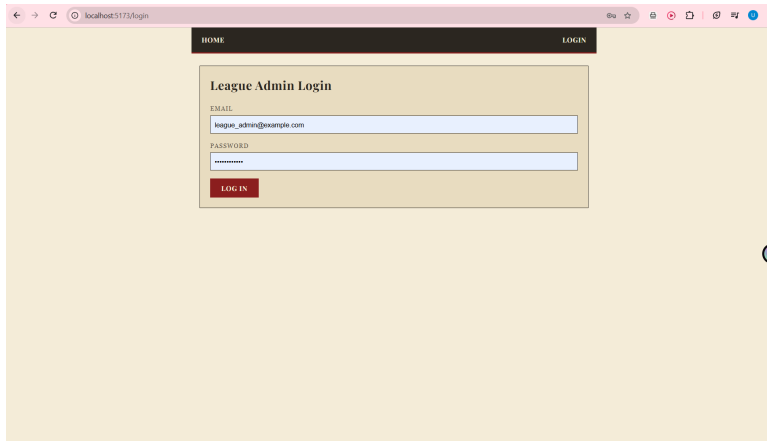


Figure 2: Login page (/login).

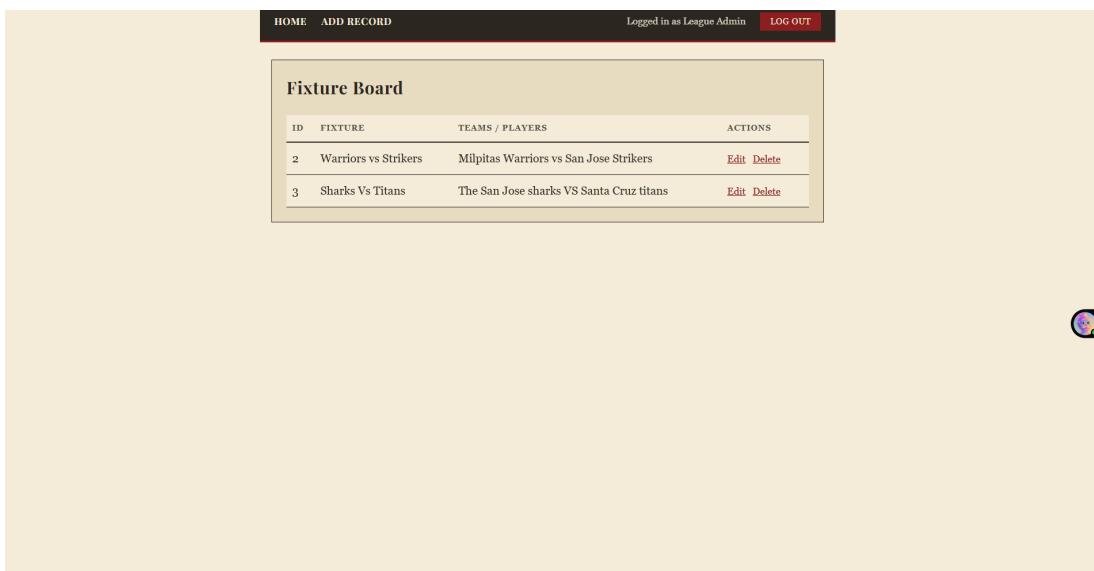
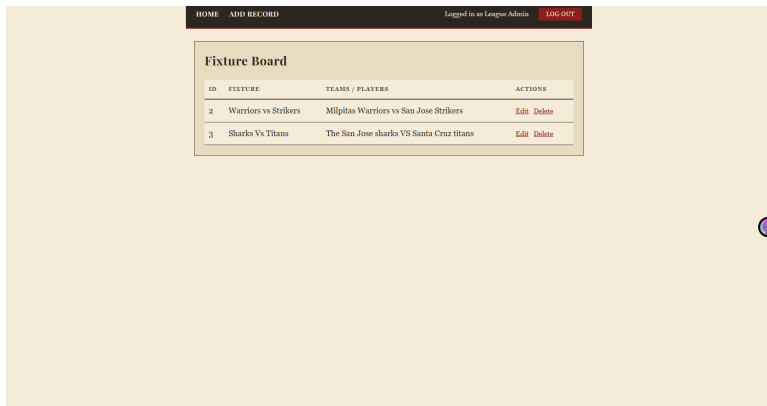


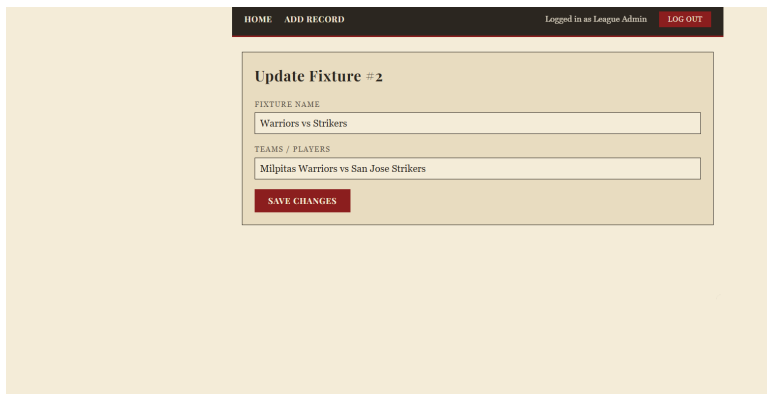
Figure 3: Home page (/), logged in – the fixture board with real data from MySQL.



The screenshot shows the Home page of a web application. At the top, there is a navigation bar with links for 'HOME' and 'ADD RECORD'. On the right side of the bar, it says 'Logged in as League Admin' and has a 'LOG OUT' button. Below the navigation bar, there is a section titled 'Fixture Board'. This section contains a table with the following data:

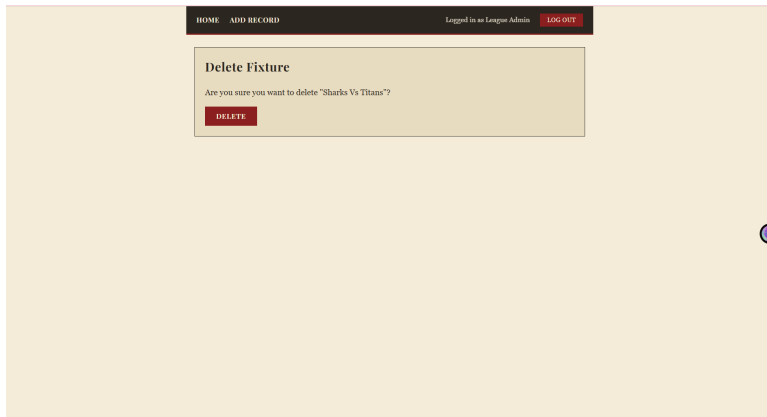
ID	FIXTURE	TEAMS / PLAYERS	ACTIONS
2	Warriors vs Strikers	Milpitas Warriors vs San Jose Strikers	Edit Delete
3	Sharks Vs Titans	The San Jose sharks VS Santa Cruz Titans	Edit Delete

Figure 4: Add Record page (/create).



The screenshot shows the 'Add Record' page, which is titled 'Update Fixture #2'. It contains a form with two input fields. The first field is labeled 'FIXTURE NAME' and contains the text 'Warriors vs Strikers'. The second field is labeled 'TEAMS / PLAYERS' and contains the text 'Milpitas Warriors vs San Jose Strikers'. Below these fields is a red button labeled 'SAVE CHANGES'.

Figure 5: Update page, showing the pre-filled form / confirmation.



The screenshot shows the 'Delete' page, which is titled 'Delete Fixture'. It contains a confirmation message: 'Are you sure you want to delete "Sharks Vs Titans"?'. Below this message is a red button labeled 'DELETE'.

Figure 6: Delete page, showing the pre-filled form / confirmation.

Part 2 – MySQL Persistence and Server-Side Sessions

Database: `s9871_rel` (PREFIX = `s9871`). SQLAlchemy models (`code/web_application/models.py`) define the primary domain table `fixtures` (auto-increment `id`, `fixture_name`, `teams_players`), plus `users` (`id`, `name`, `email` UNIQUE, `password_hash`) and `sessions` (`id` = the opaque session token itself, `user_id`, `created_at`, `expires_at`) for authentication. The connection setup (`db.py`) defines the session factory as `db_session_basede26`, the exact variable name the assignment requires.

`db.py` – database connection, required variable name

```
"""
DATA-260 Homework 4 - Part 2: MySQL database connection setup.
"""

import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

MYSQL_USER = os.environ.get("MYSQL_USER", "root")
MYSQL_PASSWORD = os.environ.get("MYSQL_PASSWORD", "")
MYSQL_HOST = "localhost"
MYSQL_DB = "s9871_rel"

DATABASE_URL = f"mysql+pymysql://{MYSQL_USER}:{MYSQL_PASSWORD}@{MYSQL_HOST}/{MYSQL_DB}"

# echo=True logs every actual SQL statement SQLAlchemy sends to MySQL -
# this will be essential later in Part 3 for literally counting N+1 queries.
engine = create_engine(DATABASE_URL, echo=True)

# Required exact variable name per the assignment.
db_session_basede26 = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def get_db():
    """FastAPI dependency: yields one DB session per request, always
    closing it afterward even if the request raised an error."""
    db = db_session_basede26()
    try:
        yield db
    finally:
        db.close()
```

`models.py` – the four tables

```
"""
DATA-260 Homework 4 - Part 2/3: SQLAlchemy table models.
"""

from datetime import datetime, timezone

from sqlalchemy import Column, Integer, String, DateTime, ForeignKey
from sqlalchemy.orm import relationship

from db import Base
```

```

class Fixture(Base):
    __tablename__ = "fixtures"

    id = Column(Integer, primary_key=True, autoincrement=True)
    fixture_name = Column(String(255), nullable=False)
    teams_players = Column(String(255), nullable=False)

    # One-directional relationship for Part 3's N+1 demonstration.
    updates = relationship("FixtureUpdate")

class FixtureUpdate(Base):
    """Part 3 test data: a related table, 200 rows, each pointing at one
    of the 5,000 fixtures. Just test data for now - a proper related
    entity with full CRUD comes in a later homework."""
    __tablename__ = "fixture_updates"

    id = Column(Integer, primary_key=True, autoincrement=True)
    fixture_id = Column(Integer, ForeignKey("fixtures.id"), nullable=False)
    note = Column(String(255), nullable=False)
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(255), nullable=False)
    email = Column(String(255), unique=True, nullable=False)
    password_hash = Column(String(255), nullable=False)

class Session(Base):
    __tablename__ = "sessions"

    id = Column(String(64), primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"), nullable=False)
    created_at = Column(DateTime, default=lambda: datetime.now(timezone.utc))
    expires_at = Column(DateTime, nullable=False)

    user = relationship("User")

```

Table definitions, straight from MySQL (mysqldump -no-data)

```

-- PLACEHOLDER: replace this file with the real reports/hw04/sql/schema.sql
-- your own machine produced via mysqldump (see RUN_LOG.txt, section 3e).
-- This placeholder reproduces the CREATE TABLE statements exactly as
-- SQLAlchemy generated them (captured with echo=True) so the report
-- still compiles and shows accurate table structures until you swap in
-- the real dump.

CREATE TABLE fixtures (
    id INTEGER NOT NULL AUTO_INCREMENT,
    fixture_name VARCHAR(255) NOT NULL,
    teams_players VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    KEY idx_fixture_name (fixture_name)
);

```

```

CREATE TABLE fixture_updates (
    id INTEGER NOT NULL AUTO_INCREMENT,
    fixture_id INTEGER NOT NULL,
    note VARCHAR(255) NOT NULL,
    created_at DATETIME,
    PRIMARY KEY (id),
    KEY fixture_id (fixture_id),
    FOREIGN KEY (fixture_id) REFERENCES fixtures (id)
);

CREATE TABLE users (
    id INTEGER NOT NULL AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    PRIMARY KEY (id),
    UNIQUE KEY (email)
);

CREATE TABLE sessions (
    id VARCHAR(64) NOT NULL,
    user_id INTEGER NOT NULL,
    created_at DATETIME,
    expires_at DATETIME NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (user_id) REFERENCES users (id)
);

```

Session design

The cookie holds only an opaque random token (`secrets.token_hex(16)`), never real data – the same principle from HW3’s session security work, carried forward: a server-side `sessions` table (now a real MySQL table, not an in-memory dict) is the only thing that makes a token valid. Passwords are hashed with `bcrypt`, never stored in plain text.

```

class LoginRequest(BaseModel):
    email: str
    password: str

def get_current_user(
    session_token: Optional[str] = Cookie(default=None),
    db: DBSession = Depends(get_db),
):
    if not session_token:
        raise HTTPException(status_code=401, detail="Not authenticated")

    session = db.query(SessionModel).filter(SessionModel.id == session_token).first()
    if not session:
        raise HTTPException(status_code=401, detail="Not authenticated")

    if session.expires_at < datetime.now(timezone.utc).replace(tzinfo=None):
        db.delete(session)
        db.commit()
        raise HTTPException(status_code=401, detail="Session expired")

    user = db.query(User).filter(User.id == session.user_id).first()
    if not user:
        raise HTTPException(status_code=401, detail="Not authenticated")
    return user

```

```

def login(payload: LoginRequest, response: Response, db: DBSession = Depends(get_db)):
    user = db.query(User).filter(User.email == payload.email).first()
    if not user or not bcrypt.checkpw(payload.password.encode(), user.password_hash.encode()):
        raise HTTPException(status_code=401, detail="Invalid email or password")

    token = secrets.token_hex(16)
    expires_at = datetime.now(timezone.utc).replace(tzinfo=None) + timedelta(hours=
SESSION_LIFETIME_HOURS)
    db.add(SessionModel(id=token, user_id=user.id, expires_at=expires_at))
    db.commit()

    response.set_cookie(
        key=SESSION_COOKIE_NAME,
        value=token,
        httponly=True,
        samesite="lax",
        secure=True,
        max_age=3600,
        path="/",
    )
    return {"id": user.id, "name": user.name, "email": user.email}

def me(user: User = Depends(get_current_user)):
    return {"id": user.id, "name": user.name, "email": user.email}

def logout(
    response: Response,
    session_token: Optional[str] = Cookie(default=None),
    db: DBSession = Depends(get_db),
):
    if session_token:
        session = db.query(SessionModel).filter(SessionModel.id == session_token).first()
        if session:
            db.delete(session)
            db.commit()
    response.delete_cookie(SESSION_COOKIE_NAME, path="/")
    return {"success": True}

```

CRUD endpoints

POST/GET/GET-by-id/PUT/DELETE on `/api/fixtures`, all requiring a valid session, enforced via a FastAPI dependency (see `get_current_user` below).

```

class FixtureCreate(BaseModel):
    fixture_name: str
    teams_players: str

class FixtureUpdate(BaseModel):
    fixture_name: str
    teams_players: str

def fixture_to_dict(f: Fixture):
    return {"id": f.id, "fixture_name": f.fixture_name, "teams_players": f.teams_players}

def list_fixtures(user: User = Depends(get_current_user), db: DBSession = Depends(get_db)):
    fixtures = db.query(Fixture).all()
    return [fixture_to_dict(f) for f in fixtures]

```

```

def get_fixture(fixture_id: int, user: User = Depends(get_current_user), db: DBSession = Depends(
    get_db)):
    fixture = db.query(Fixture).filter(Fixture.id == fixture_id).first()
    if not fixture:
        raise HTTPException(status_code=404, detail="Fixture not found")
    return fixture_to_dict(fixture)

def create_fixture(payload: FixtureCreate, user: User = Depends(get_current_user), db: DBSession =
    Depends(get_db)):
    fixture = Fixture(fixture_name=payload.fixture_name, teams_players=payload.teams_players)
    db.add(fixture)
    db.commit()
    db.refresh(fixture)
    return fixture_to_dict(fixture)

def update_fixture(fixture_id: int, payload: FixtureUpdate, user: User = Depends(get_current_user), db
    : DBSession = Depends(get_db)):
    fixture = db.query(Fixture).filter(Fixture.id == fixture_id).first()
    if not fixture:
        raise HTTPException(status_code=404, detail="Fixture not found")
    fixture.fixture_name = payload.fixture_name
    fixture.teams_players = payload.teams_players
    db.commit()
    db.refresh(fixture)
    return fixture_to_dict(fixture)

def delete_fixture(fixture_id: int, user: User = Depends(get_current_user), db: DBSession = Depends(
    get_db)):
    fixture = db.query(Fixture).filter(Fixture.id == fixture_id).first()
    if not fixture:
        raise HTTPException(status_code=404, detail="Fixture not found")
    db.delete(fixture)
    db.commit()
    return {"success": True}

```

Postman screenshots – one per operation

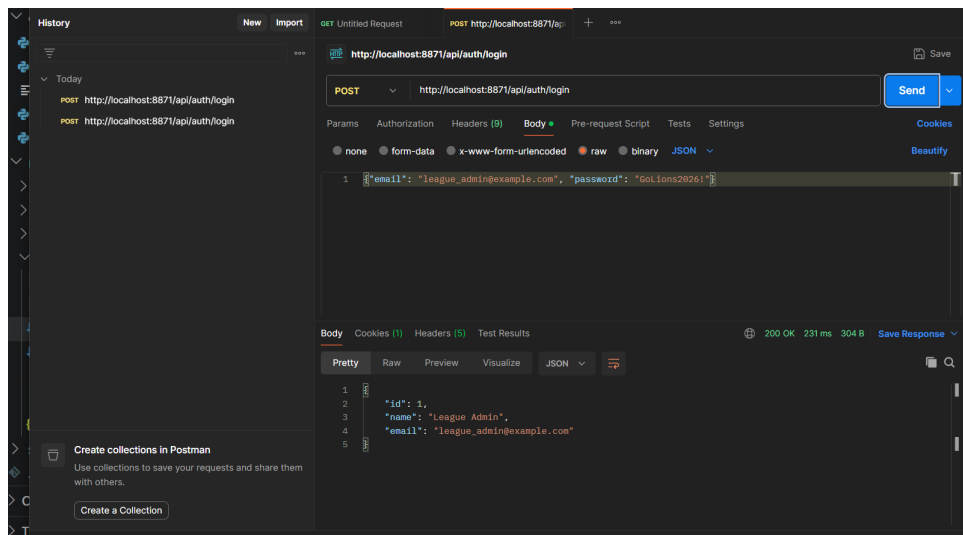


Figure 6: POST `/api/auth/login` – 200, sets the session cookie.

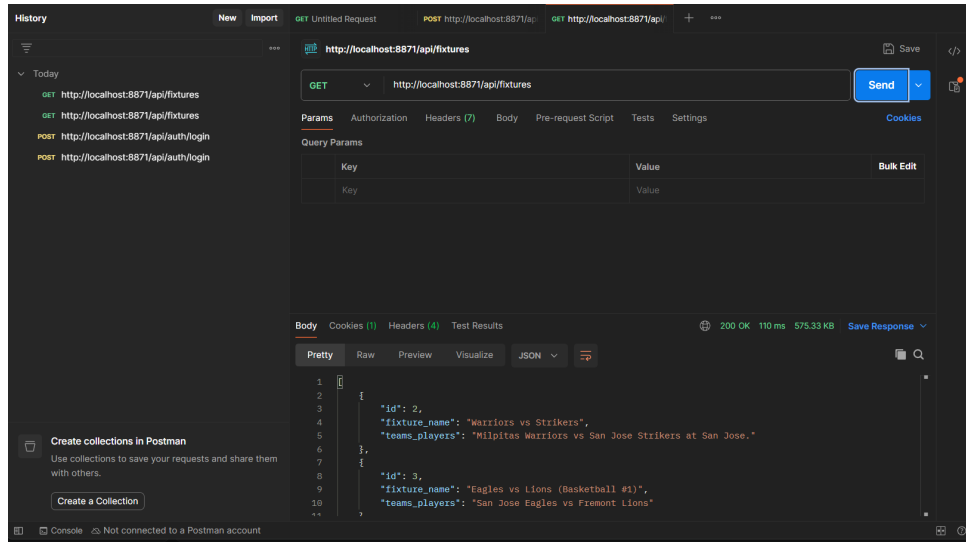


Figure 7: GET /api/fixtures – 200, the full record list.

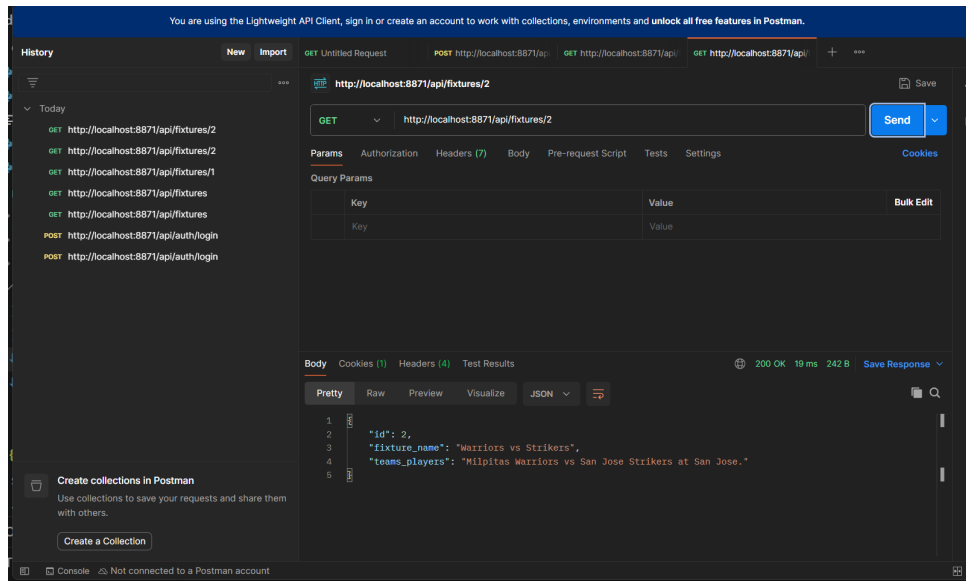


Figure 8: GET /api/fixtures/{id} – 200, a single record.

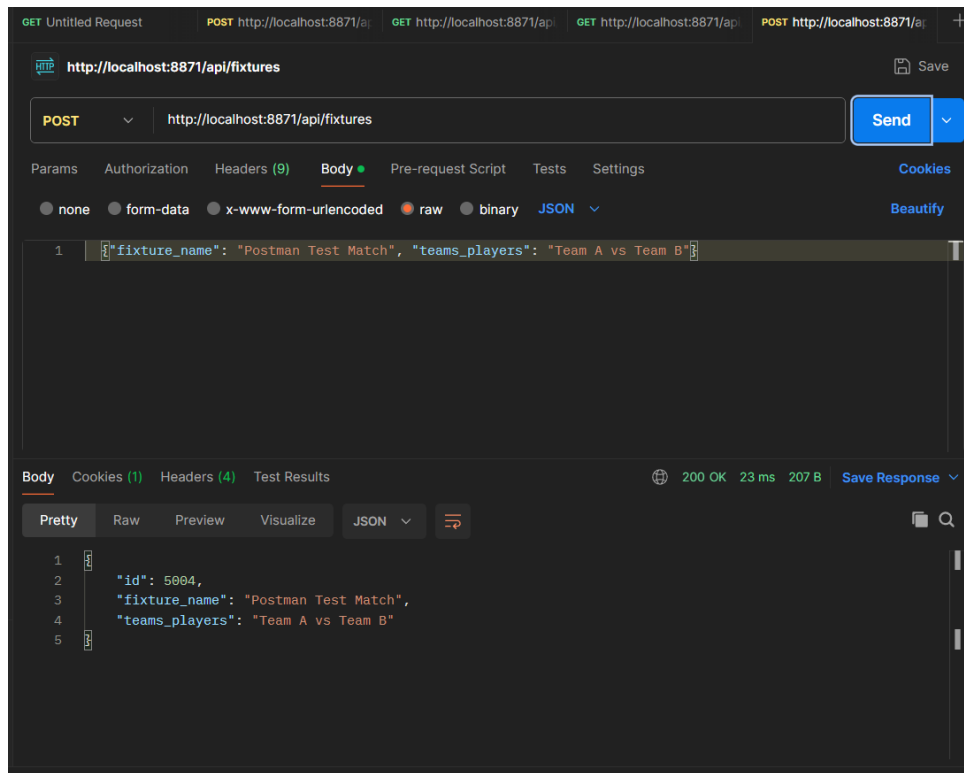


Figure 9: `POST /api/fixtures` – 200, a new record with an auto-incremented id.

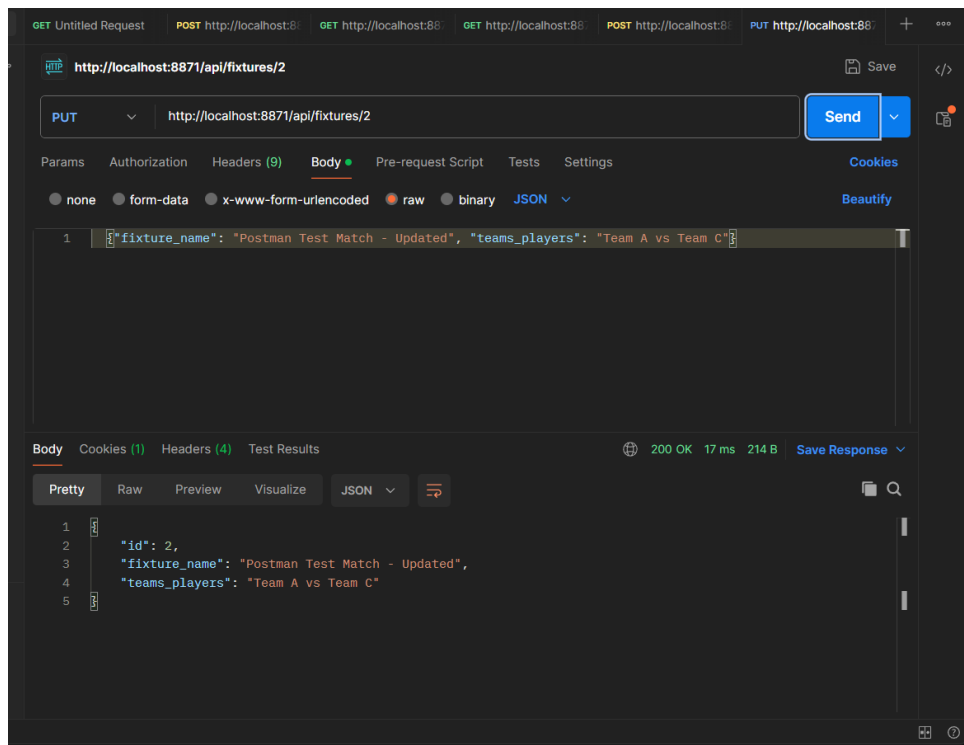


Figure 10: `PUT /api/fixtures/{id}` – 200, the updated record.

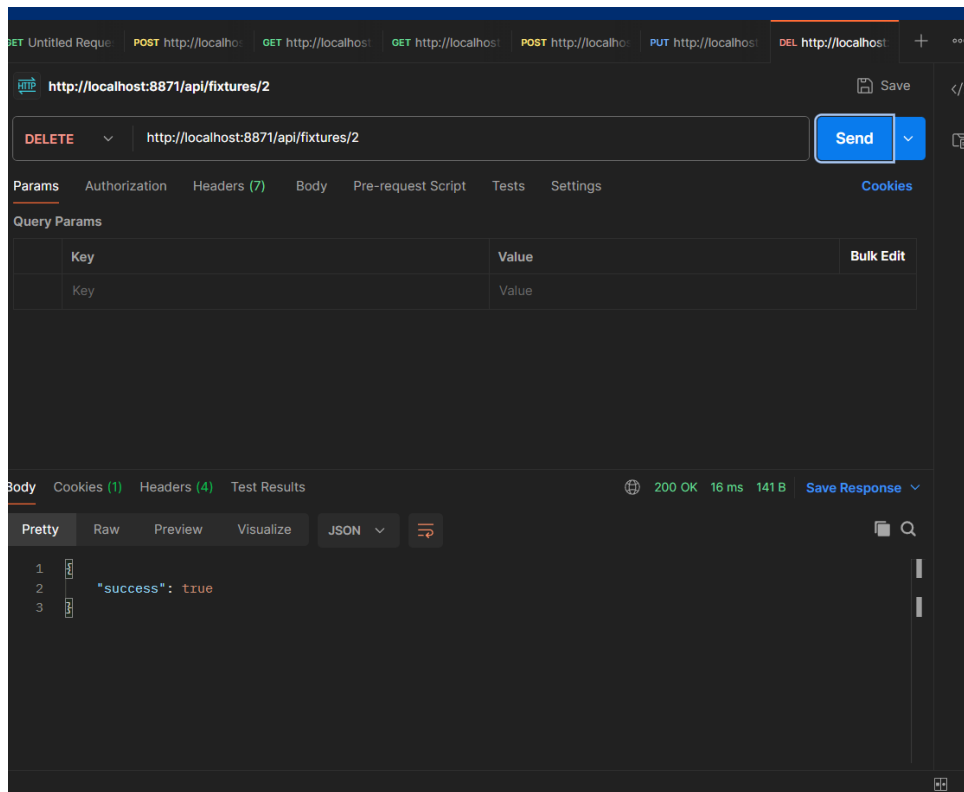


Figure 11: DELETE /api/fixtures/{id} – 200.

Database screenshot

Real output from the `mysql` command-line client against `s9871_rel`, showing real rows (10 of the seeded fixtures, the one real user, and the count of related rows from Part 3):

```
(venv) PS C:\Users\patel\data260-9871> & "C:\Program Files\MySQL\MySQL Server 8.0\bin\mysql.exe" -u hw4app -p
s9871_rel -e "SELECT * FROM fixtures LIMIT 10; SELECT id, name, email FROM users; SELECT COUNT(*) AS
fixture_updates_count FROM fixture_updates;"
Enter password: *****
+-----+-----+-----+-----+
| id | fixture_name | teams_players |
+-----+-----+-----+-----+
| 3 | Eagles vs Lions (Basketball #1) | San Jose Eagles vs Fremont Lions |
| 4 | Strikers vs Hawks (Basketball #2) | San Jose Strikers vs Santa Clara Hawks |
| 5 | Wolves vs Lions (Basketball #3) | San Jose Wolves vs Sunnyvale Lions |
| 6 | Lions vs Strikers (Soccer #4) | Milpitas Lions vs Santa Clara Strikers |
| 7 | Bears vs Sharks (Soccer #5) | Sunnyvale Bears vs Oakland Sharks |
| 8 | Falcons vs Titans (Volleyball #6) | Oakland Falcons vs Milpitas Titans |
| 9 | Lions vs Warriors (Soccer #7) | Sunnyvale Lions vs Fremont Warriors |
| 10 | Bears vs Panthers (Volleyball #8) | Sunnyvale Bears vs Fremont Panthers |
| 11 | Bears vs Wolves (Volleyball #9) | Milpitas Bears vs Sunnyvale Wolves |
| 12 | Strikers vs Bears (Volleyball #10) | Oakland Strikers vs Santa Clara Bears |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| id | name | email |
+-----+-----+-----+-----+
| 1 | League Admin | league_admin@example.com |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| fixture_updates_count |
+-----+-----+-----+-----+
| 200 |
+-----+-----+-----+-----+
```

Project folder structure

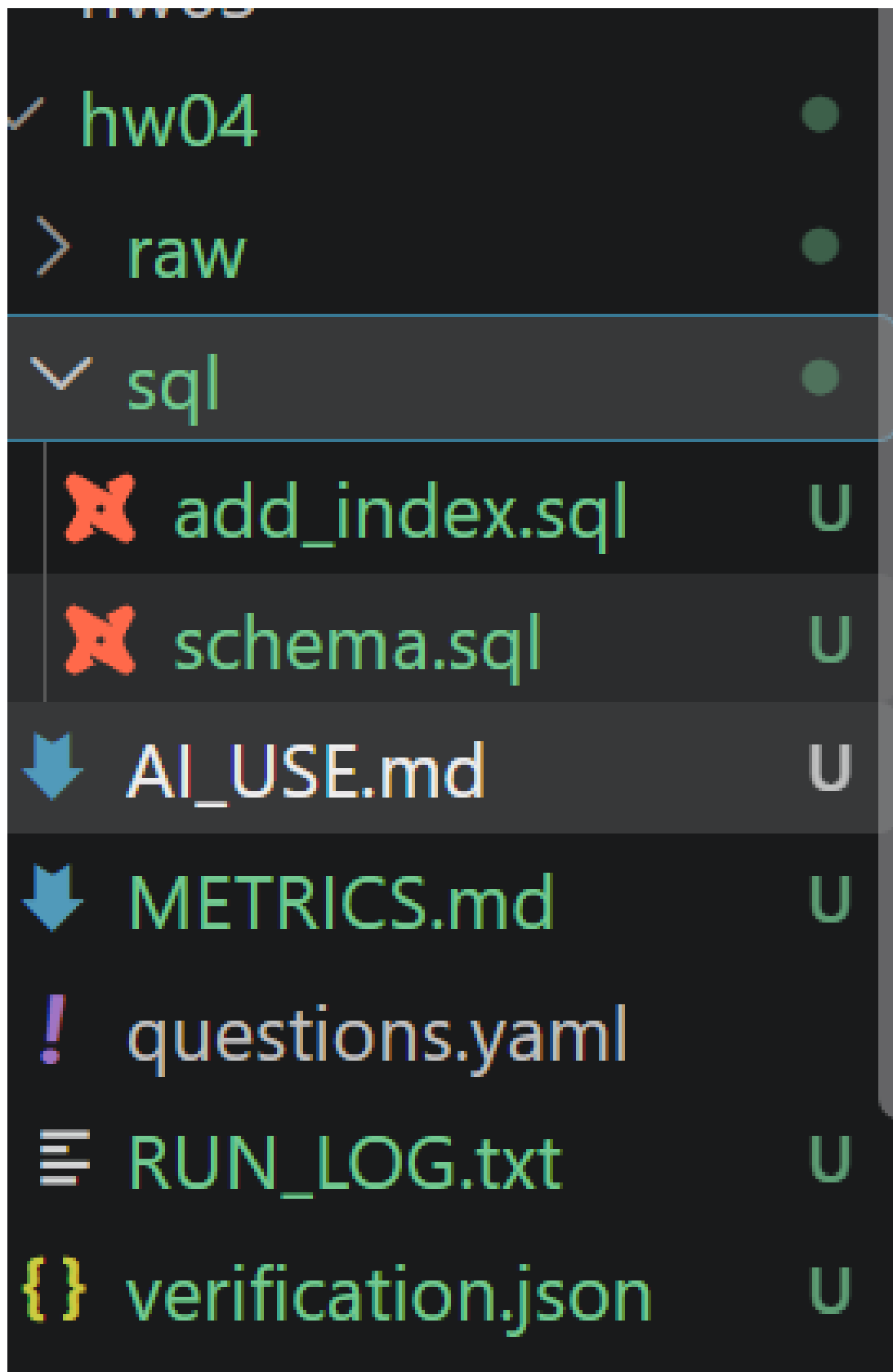


Figure 12: Project folder structure.

Part 3 – N+1 Measurement and Query Tuning

A related test table, `fixture_updates` (`code/web_application/models.py`), was seeded with 5,000 fixtures and 200 related rows using `SEED = 9871`.

```
"""
DATA-260 Homework 4 - Part 3: Seed 5,000 fixtures + 200 related
fixture_updates rows, using SEED for reproducibility.

Run with: python seed_n1_data.py
"""

import random

from db import Base, engine, db_session_based26
import models
from models import Fixture, FixtureUpdate

SEED = 9871
NUM_FIXTURES = 5000
NUM_UPDATES = 200

TEAM_NAMES = [
    "Lions", "Tigers", "Warriors", "Strikers", "Falcons", "Titans",
    "Eagles", "Sharks", "Wolves", "Bears", "Hawks", "Panthers",
]
CITIES = ["Milpitas", "San Jose", "Fremont", "Santa Clara", "Sunnyvale", "Oakland"]
SPORTS = ["Soccer", "Basketball", "Volleyball", "Cricket"]

def main():
    random.seed(SEED)
    Base.metadata.create_all(bind=engine)

    db = db_session_based26()
    try:
        existing = db.query(Fixture).count()
        print(f"Existing fixtures before seeding: {existing}")

        print(f"Seeding {NUM_FIXTURES} fixtures...")
        fixture_ids = []
        batch = []
        for i in range(NUM_FIXTURES):
            team_a, team_b = random.sample(TEAM_NAMES, 2)
            city_a, city_b = random.sample(CITIES, 2)
            sport = random.choice(SPORTS)
            fixture = Fixture(
                fixture_name=f"{team_a} vs {team_b} ({sport} #{i+1})",
                teams_players=f"{city_a} {team_a} vs {city_b} {team_b}",
            )
            batch.append(fixture)
            if len(batch) >= 500:
                db.add_all(batch)
                db.commit()
                batch = []
        if batch:
            db.add_all(batch)
            db.commit()

        # Re-fetch actual ids assigned by MySQL (auto-increment)
```

```

all_ids = [row.id for row in db.query(Fixture.id).all()]
print(f"Total fixtures now in DB: {len(all_ids)}")

print(f"Seeding {NUM_UPDATES} related fixture_updates rows...")
update_batch = []
for i in range(NUM_UPDATES):
    fid = random.choice(all_ids)
    update_batch.append(FixtureUpdate(
        fixture_id=fid,
        note=f"Update note #{i+1}: schedule confirmed.",
    ))
db.add_all(update_batch)
db.commit()

total_fixtures = db.query(Fixture).count()
total_updates = db.query(FixtureUpdate).count()
print(f"\nDone. fixtures={total_fixtures}, fixture_updates={total_updates}")
finally:
    db.close()

if __name__ == "__main__":
    main()

```

The naive endpoint (intentional N+1) and the fixed endpoint (eager loading)

`/api/fixtures-naive` runs one query for the page, then one more query per fixture to fetch its related rows: `1 + page_size` queries. `/api/fixtures-fixed` uses SQLAlchemy's `selectinload` to fetch every related row in a single second query: always exactly 2 queries, regardless of page size.

```

def fixture_with_updates_to_dict(f: Fixture):
    return {
        "id": f.id,
        "fixture_name": f.fixture_name,
        "teams_players": f.teams_players,
        "updates": [u.note for u in f.updates],
    }

def list_fixtures_naive(
    request: Request,
    page_size: int = 10,
    user: User = Depends(get_current_user),
    db: DBSession = Depends(get_db),
):
    """Intentionally naive: fetches the page of fixtures with ONE query,
    then fires ONE ADDITIONAL query per fixture to fetch its related
    updates - the classic N+1 problem. Total queries = 1 + page_size."""
    # Reset the counter explicitly - Connection.info persists at the
    # pooled-connection level across checkouts, so it must be zeroed
    # here rather than assumed to start fresh each request.
    db.connection().info["query_count"] = 0
    fixtures = db.query(Fixture).limit(page_size).all()
    results = []
    for f in fixtures:
        updates = db.query(FixtureUpdate).filter(FixtureUpdate.fixture_id == f.id).all()
        results.append({
            "id": f.id,
            "fixture_name": f.fixture_name,
            "teams_players": f.teams_players,

```

```

        "updates": [u.note for u in updates],
    })
    request.state.sql_query_count = db.connection().info.get("query_count", 0)
    return results

def list_fixtures_fixed(
    request: Request,
    page_size: int = 10,
    user: User = Depends(get_current_user),
    db: DBSession = Depends(get_db),
):
    """Fixed: uses selectinload to eager-load each fixture's related
    updates in a SECOND single query (using WHERE fixture_id IN (...)),
    instead of one query per fixture. Total queries = 2, regardless of
    page_size."""
    db.connection().info["query_count"] = 0
    fixtures = db.query(Fixture).options(selectinload(Fixture.updates)).limit(page_size).all()
    results = [fixture_with_updates_to_dict(f) for f in fixtures]
    request.state.sql_query_count = db.connection().info.get("query_count", 0)
    return results

```

Counting the real SQL queries per request

A middleware counts every SQL statement executed on a given database connection during one request (via SQLAlchemy's `before_cursor_execute` event) and returns it as the **X-SQL-Query-Count** response header, visible in the Postman screenshots below. Two bugs were found and fixed while building this (see `AI_USE.md`): a `contextvar` that did not survive FastAPI's sync-endpoint thread boundary, and `Connection.info` persisting across pooled-connection reuse rather than resetting per request.

```

"""
DATA-260 Homework 4 - Part 3: SQL query counting.

Counts the actual SQL statements executed on a given database connection
during one request, via SQLAlchemy's before_cursor_execute event, storing
the count on that Connection's own .info dict (scoped per connection
checkout - i.e., per request, since each request's Session uses one
connection for its duration). The count is then smuggled out to the
middleware via request.state, which is safe because it's a plain shared
object reference passed explicitly through the ASGI call chain - NOT a
contextvar, which (as discovered the hard way) does not propagate
mutations made inside FastAPI's separate sync-endpoint thread back to
the async middleware's copy of the context.
"""

from sqlalchemy import event

def register_query_counter(engine):
    @event.listens_for(engine, "before_cursor_execute")
    def _count_query(conn, cursor, statement, parameters, context, executemany):
        conn.info["query_count"] = conn.info.get("query_count", 0) + 1

async def query_count_middleware(request, call_next):
    response = await call_next(request)
    count = getattr(request.state, "sql_query_count", None)
    if count is not None:
        response.headers["X-SQL-Query-Count"] = str(count)

```

```
return response
```

Measurement script (30 requests × 3 page sizes × 2 versions = 180 requests)

```
"""
DATA-260 Homework 4 - Part 3: N+1 measurement script.

Fires 30 real HTTP requests at each of 3 page sizes (10, 50, 200)
against BOTH the naive and fixed endpoints (180 total requests),
recording the real SQL query count (from the X-SQL-Query-Count header
our middleware attaches) and latency for each one.

Run with: python measure_n1.py
"""

import csv
import json
import statistics
import time
from pathlib import Path

import requests

BASE_URL = "http://localhost:8871"
EMAIL = "league_admin@example.com"
PASSWORD = "GoLions2026!"
PAGE_SIZES = [10, 50, 200]
VERSIONS = ["naive", "fixed"]
RUNS_PER_COMBO = 30

REPO_ROOT = Path(__file__).resolve().parent.parent.parent
RAW_DIR = REPO_ROOT / "reports" / "hw04" / "raw"
RAW_DIR.mkdir(parents=True, exist_ok=True)

def percentile(values, pct):
    values_sorted = sorted(values)
    k = (len(values_sorted) - 1) * (pct / 100)
    f = int(k)
    c = min(f + 1, len(values_sorted) - 1)
    if f == c:
        return values_sorted[f]
    return values_sorted[f] + (values_sorted[c] - values_sorted[f]) * (k - f)

def main():
    session = requests.Session()

    print("Logging in...")
    resp = session.post(f"{BASE_URL}/api/auth/login", json={"email": EMAIL, "password": PASSWORD})
    resp.raise_for_status()
    print(f"  Logged in as: {resp.json()['name']}")

    # requests' cookie jar correctly honors the Secure flag (unlike real
    # browsers, which treat localhost as a special trustworthy exception -
    # see HW3's cookie security work) - so it won't automatically resend
    # our Secure cookie over this plain http:// connection. We bypass that
    # by passing the cookie explicitly on every request instead of relying
    # on the session's automatic cookie jar.
```

```

session_token = session.cookies.get("session_token")
auth_cookies = {"session_token": session_token}

all_rows = []

for version in VERSIONS:
    for page_size in PAGE_SIZES:
        print(f"\nMeasuring: version={version}, page_size={page_size}")
        for run in range(1, RUNS_PER_COMBO + 1):
            start = time.perf_counter()
            resp = session.get(
                f"{BASE_URL}/api/fixtures-{version}",
                params={"page_size": page_size},
                cookies=auth_cookies,
            )
            latency_ms = (time.perf_counter() - start) * 1000
            resp.raise_for_status()

            query_count = int(resp.headers.get("X-SQL-Query-Count", -1))
            num_results = len(resp.json())

            row = {
                "version": version,
                "page_size": page_size,
                "run": run,
                "sql_query_count": query_count,
                "latency_ms": round(latency_ms, 2),
                "num_results": num_results,
            }
            all_rows.append(row)

            if run == 1 or run == RUNS_PER_COMBO:
                print(f"  run {run:2d}: queries={query_count} latency={latency_ms:.2f}ms results
={num_results}")

# Save raw data (all 180 rows)
raw_json_path = RAW_DIR / "n1_measurements.json"
with open(raw_json_path, "w", encoding="utf-8") as f:
    json.dump(all_rows, f, indent=2)

raw_csv_path = RAW_DIR / "n1_measurements.csv"
with open(raw_csv_path, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["version", "page_size", "run", "sql_query_count", "
latency_ms", "num_results"])
    writer.writeheader()
    for row in all_rows:
        writer.writerow(row)

print(f"\n\nRaw data ({len(all_rows)} rows) saved to:")
print(f"  {raw_json_path}")
print(f"  {raw_csv_path}")

# Compute summary table
summary = []
for version in VERSIONS:
    for page_size in PAGE_SIZES:
        group = [r for r in all_rows if r["version"] == version and r["page_size"] == page_size]
        latencies = [r["latency_ms"] for r in group]
        query_counts = set(r["sql_query_count"] for r in group)
        summary.append({

```

```

        "page_size": page_size,
        "version": version,
        "sql_stmts_per_req": query_counts.pop() if len(query_counts) == 1 else f"varied: {
query_counts}",
        "p50_ms": round(percentile(latencies, 50), 2),
        "p95_ms": round(percentile(latencies, 95), 2),
        "p99_ms": round(percentile(latencies, 99), 2),
    })

summary_path = RAW_DIR / "n1_summary_table.json"
with open(summary_path, "w", encoding="utf-8") as f:
    json.dump(summary, f, indent=2)

print(f"\n{'Page size':<10}{ 'Version':<10}{ 'SQL stmts/req':<15}{ 'p50 (ms)':<11}{ 'p95 (ms)':<11}{ 'p99 (ms)':<11}")
print("-" * 68)
for row in summary:
    print(f"{row['page_size']:<10}{row['version']:<10}{str(row['sql_stmts_per_req']):<15}"
          f"{row['p50_ms']:<11}{row['p95_ms']:<11}{row['p99_ms']:<11}")

print(f"\nSummary saved to: {summary_path}")

# Speedup comparison
print(f"\n{'='*68}")
print("Speedup (naive p50 / fixed p50) per page size:")
for page_size in PAGE_SIZES:
    naive_row = next(r for r in summary if r["version"] == "naive" and r["page_size"] == page_size
    )
    fixed_row = next(r for r in summary if r["version"] == "fixed" and r["page_size"] == page_size
    )
    speedup = naive_row["p50_ms"] / fixed_row["p50_ms"] if fixed_row["p50_ms"] > 0 else float("inf
    ")
    print(f"  page_size={page_size}: naive={naive_row['p50_ms']}ms, fixed={fixed_row['p50_ms']}ms,
    speedup={speedup:.2f}x")

if __name__ == "__main__":
    main()

```

Results

Full details, the speed-up explanation and the caveats are in `METRICS.md`; the core table:

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	11	12.02	17.42	19.35
10	fixed	2	8.15	10.83	12.49
50	naive	51	37.36	43.99	47.46
50	fixed	2	9.91	11.35	16.92
200	naive	201	174.56	219.36	228.14
200	fixed	2	14.43	17.28	40.84

Reported in `METRICS.md`: naive's median latency grows almost linearly with page size (about 0.6–0.9 ms per extra row) because every extra row is another round trip to MySQL; fixed's cost is nearly flat (about 0.03–0.04 ms per extra row) because the query count never changes. The speed-up (naive / fixed) therefore grows with page size: 1.47x, 3.77x, 12.10x.

Index and EXPLAIN

MySQL/InnoDB had already auto-created an index on `fixture_updates.fixture_id` to enforce the foreign key, so a second index there would show no change. The one index added instead targets `fixtures.fixture_name`, a natural real-world search/filter column that was still unindexed:

```
-- DATA-260 Homework 4, Part 3, requirement 8: the one index added.
-- Database: s9871_rel
--
-- Query used for the EXPLAIN comparison:
--   EXPLAIN SELECT * FROM fixtures WHERE fixture_name = 'Lions vs Tigers';
--
-- BEFORE the index:
--   type=ALL possible_keys=NULL key=NULL key_len=NULL ref=NULL
--   rows=4856 filtered=10.00 Extra=Using where      (full table scan)
--
-- AFTER the index:
--   type=ref possible_keys=idx_fixture_name key=idx_fixture_name
--   key_len=1022 ref=const rows=1 filtered=100.00 Extra=NULL      (index lookup)
--
-- Applied with the mysql command-line client:
ALTER TABLE fixtures ADD INDEX idx_fixture_name (fixture_name);

-- To reproduce the "before" plan, drop the index and re-run the EXPLAIN:
-- ALTER TABLE fixtures DROP INDEX idx_fixture_name;
```

```
[2026-09-28 11:49:10] EXPLAIN before the index
> EXPLAIN SELECT * FROM fixtures WHERE fixture_name = 'Lions vs Tigers';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | fixtures | ALL | NULL | NULL | NULL | NULL | 4856 | 10.00 | Using where |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

[2026-09-28 11:49:10] EXPLAIN after the index
> EXPLAIN SELECT * FROM fixtures WHERE fixture_name = 'Lions vs Tigers';
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | select_type | table | type | possible_keys | key | key_len | ref | rows | filtered | Extra |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1 | SIMPLE | fixtures | ref | idx_fixture_name | idx_fixture_name | 1022 | const | 1 | 100.00 | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Before the index, MySQL had to scan an estimated 4,856 of the table's rows (`type=ALL`) to find a match. After it, MySQL jumps straight to the single matching row via the index (`type=ref`).

Postman screenshots – naive vs. fixed at each page size

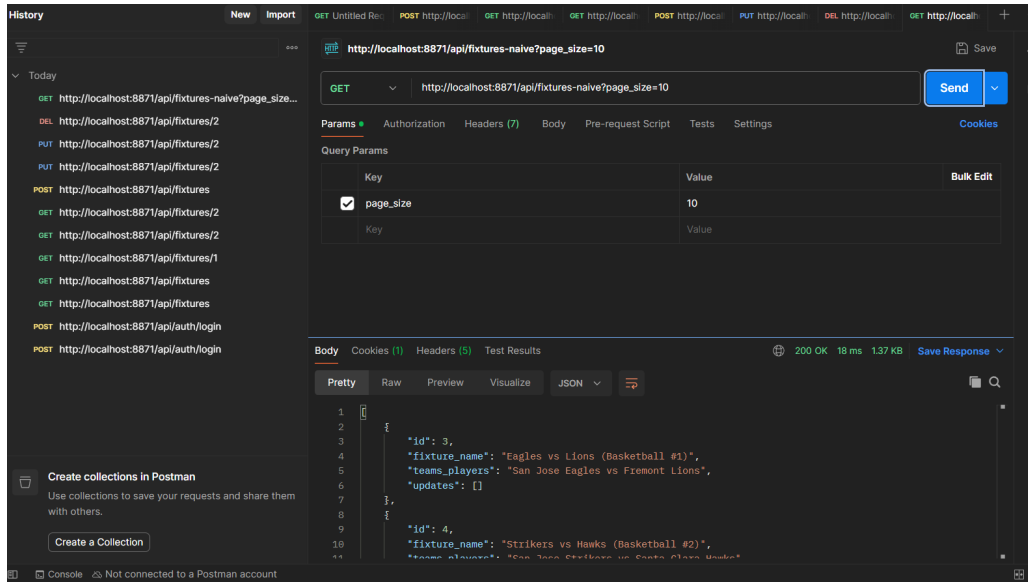


Figure 13: `GET /api/fixtures-naive?page_size=10` – note the `X-SQL-Query-Count` response header.

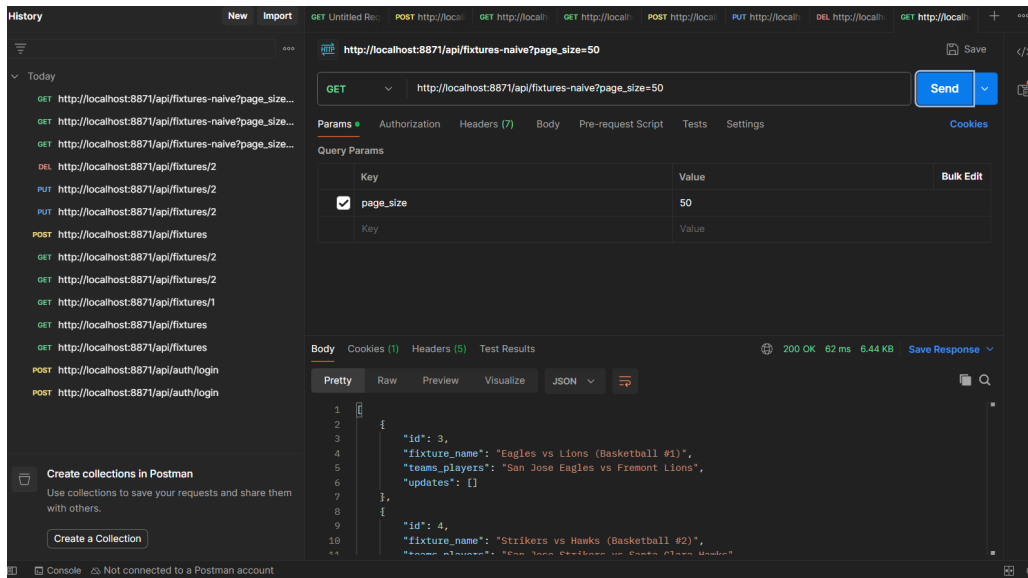


Figure 14: `GET /api/fixtures-naive?page_size=50` – note the `X-SQL-Query-Count` response header.

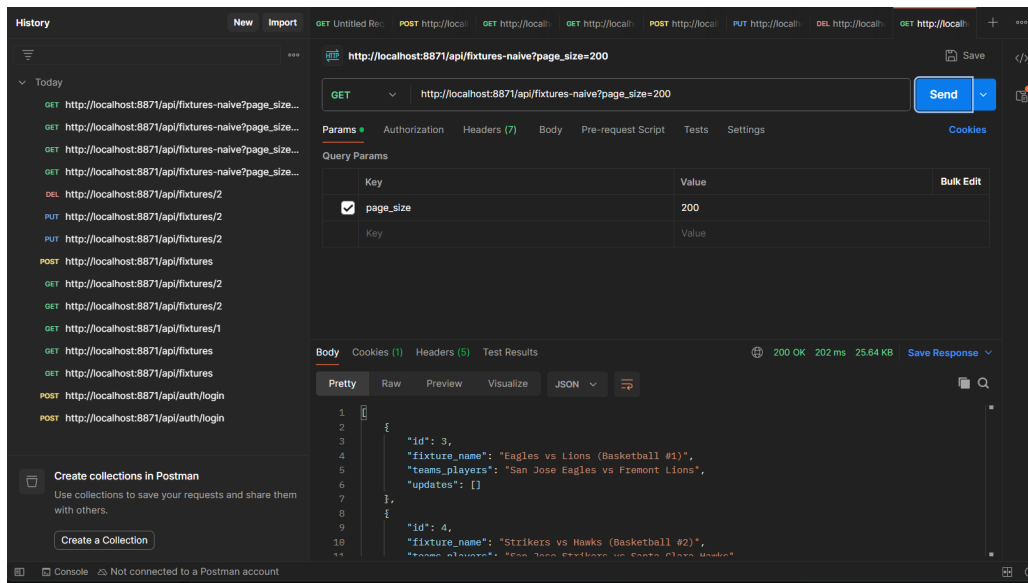


Figure 15: *GET /api/fixtures-naive?page_size=200* – note the *X-SQL-Query-Count* response header.

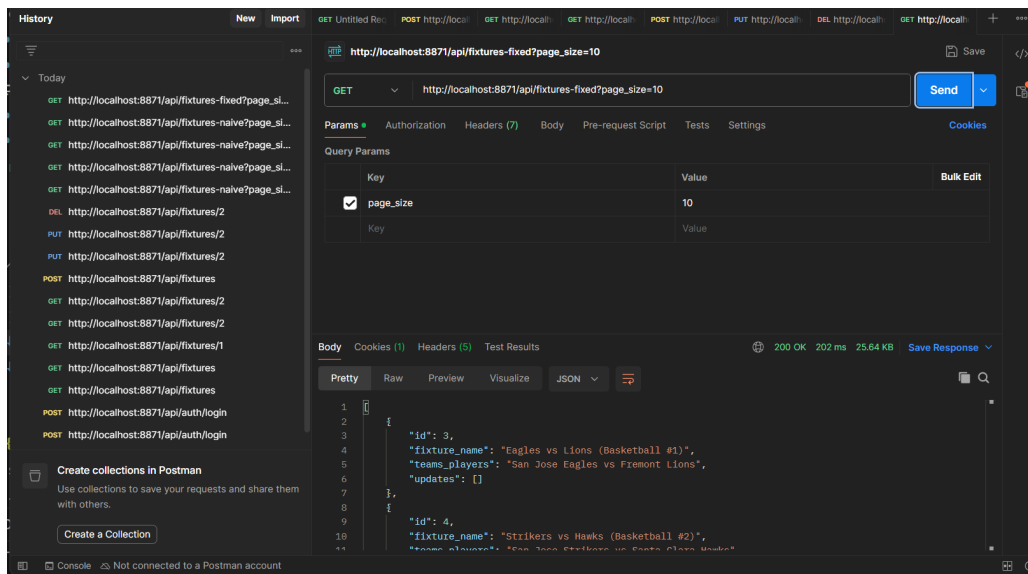


Figure 16: *GET /api/fixtures-fixed?page_size=10* – note the *X-SQL-Query-Count* response header.

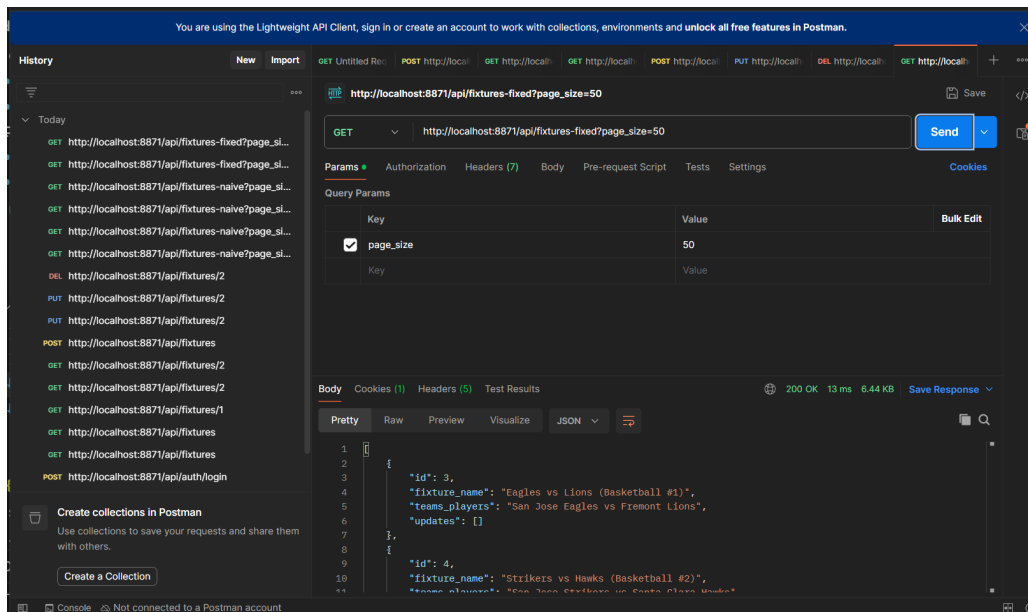


Figure 17: *GET /api/fixtures-fixed?page_size=50* – note the *X-SQL-Query-Count* response header.

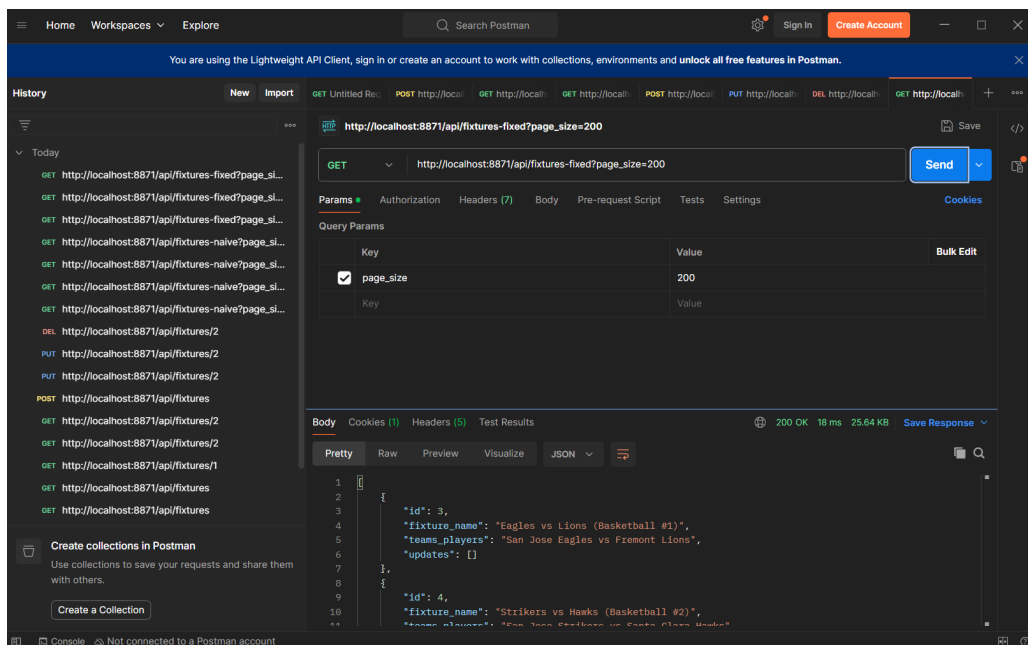


Figure 18: *GET /api/fixtures-fixed?page_size=200* – note the *X-SQL-Query-Count* response header.

Part 4 – Grounded RAG Question-Answering System

This section covers the seven requirements of Part 4 in order. All output below is real terminal output from `python rag.py` and `python rescore.py`. Where a long answer from configuration A is shortened, the cut is marked, and the complete output is saved in `reports/hw04/raw/run_transcript.txt`.

Settings used for every run

Item	Value
Corpus	6 public league rulebooks reused from HW3 (<code>reports/hw03/corpus/</code>); 489 chunks
Chunking	at most 500 characters, 50-character overlap, cut on word boundaries
Embeddings / store	<code>sentence-transformers/all-MiniLM-L6-v2</code> via <code>LlamaIndex VectorStoreIndex</code> (cosine similarity)
Local model	<code>qwen3:8b</code> through <code>Ollama</code> and <code>src/model_client.py</code> , temperature 0.0
Retrieval settings	<code>top_k = 3</code> (sweep: 1, 3, 5); relevance cutoff 0.30; duplicate threshold 0.80
Full run	9/28/2026, about 10:51–11:03 AM local time (36 model calls)
Re-scoring	9/28/2026, about 11:17 AM (<code>rescore.py</code> , no model calls)
Questions fixed	committed before the run: commit <code>90703f3</code>

4.1 Corpus and index (requirement 1)

The six rulebooks are tidied (odd spaces and line breaks), cut into chunks of at most 500 characters with a 50-character overlap, and each chunk is stored with its text, its source file name and a `chunk_id` such as `cricket_uspl_t20_rules::0029`. The chunks are embedded and loaded into a `LlamaIndex VectorStoreIndex`. The file name and id are kept out of the text that gets embedded, so they cannot change the scores. Sizes are counted in characters: the embedding model only reads about 256 word-pieces, so a 500-character chunk fits inside it.

Code:

```
CHUNK_SIZE = 500
CHUNK_OVERLAP = 50

def normalize_text(text):
    """Tidy whitespace (the PDFs contain non-breaking spaces, CRLFs, page-break
    runs) so character counts mean something."""
    text = text.replace("\r\n", "\n").replace("\r", "\n")
    text = re.sub(r"[\t\u00a0\u2002\u2003\u2009]+", " ", text)
    text = re.sub(r" ?\n ?", "\n", text)
    text = re.sub(r"\n{3,}", "\n\n", text)
    return text.strip()

def chunk_text(text, size=CHUNK_SIZE, overlap=CHUNK_OVERLAP):
    """Sliding window of at most 'size' characters, consecutive windows sharing
    about 'overlap' characters. Boundaries snap to whitespace so no word is cut
    in half."""
    if overlap >= size:
        raise ValueError("overlap must be smaller than size")
    chunks, n, start = [], len(text), 0
    while start < n:
        end = min(start + size, n)
        if end < n:
            window = text[start:end]
            cut = max(window.rfind(" "), window.rfind("\n"))
            if cut > size // 2:
                end = start + cut
            piece = text[start:end].strip()
            if piece:
                chunks.append(piece)
        if end >= n:
            break
        nxt = max(end - overlap, start + 1)
        while nxt < end and not text[nxt - 1].isspace():
            nxt += 1
            # start the next chunk on a word boundary
        start = nxt if nxt > start else end
    return chunks
```

```

def build_nodes(corpus_dir, size, overlap):
    """One TextNode per chunk, carrying text, source file name and chunk_id.
    Metadata is excluded from BOTH the embedding text and the LLM text, so the
    file name cannot leak into retrieval scores."""
    nodes, per_doc = [], {}
    for path in sorted(Path(corpus_dir).glob("*.txt")):
        raw = path.read_text(encoding="utf-8", errors="replace")
        chunks = chunk_text(normalize_text(raw), size, overlap)
        per_doc[path.name] = {"characters": len(raw), "chunks": len(chunks)}
        for i, piece in enumerate(chunks):
            cid = f"{path.stem}:{i:04d}"
            node = TextNode(text=piece, id=cid,
                            metadata={"source": path.name, "chunk_id": cid})
            node.excluded_embed_metadata_keys = ["source", "chunk_id"]
            node.excluded_llm_metadata_keys = ["source", "chunk_id"]
            nodes.append(node)
    if len(per_doc) < 5:
        raise SystemExit(f"Need at least 5 documents in {corpus_dir}, found {len(per_doc)}")
    return nodes, per_doc

def get_embed_model():
    from llama_index.embeddings.huggingface import HuggingFaceEmbedding
    return HuggingFaceEmbedding(model_name=EMBED_MODEL_NAME)

index = VectorStoreIndex(nodes, embed_model=embed_model or get_embed_model())

```

Output:

```

=== Corpus and index ===
basketball_las_vegas_youth_rules.txt           36842 chars    83 chunks
cricket_uspl_t20_rules.txt                     60266 chars   132 chunks
general_douglas_county_all_sports_rules.txt    11980 chars    26 chunks
soccer_ayso_basic_rules.txt                    66166 chars   145 chunks
soccer_ayso_national_rules.txt                 25334 chars    53 chunks
volleyball_fort_collins_manual.txt             23426 chars    50 chunks
total: 6 documents, 489 chunks (chunk_size=500, overlap=50 characters)
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate
limits and faster downloads.
[model-loading progress bar omitted]

```

4.2 Retrieval printed before the LLM call (requirement 2)

For every question the top 3 chunks are printed with source, chunk id, cosine score and the start of the text, before any model is called. This keeps a retrieval problem separate from a generation problem. Notice that Q6 (unrelated) scores 0.13 to 0.15, while every other question scores 0.58 or higher, and that Q5's chunks (0.61 to 0.64) look as relevant as the real answers to Q2 even though none of them can answer an MLS question.

Code:

```

def retrieve(index, question, k):
    """Top-k chunks as plain dicts (rank, source, chunk_id, cosine score, text)."""
    results = index.as_retriever(similarity_top_k=k).retrieve(question)
    return [{
        "rank": rank,
        "source": r.node.metadata["source"],
        "chunk_id": r.node.metadata["chunk_id"],
        "score": float(r.score),
        "text": r.node.get_content(),
    } for rank, r in enumerate(results, 1)]

# ---- Step 2: retrieval, printed before any LLM call -----
emit("\n=== Retrieval (top_k=3), printed before calling the LLM ===")
retrieval = {}
for q in questions:

```

```

chunks = retrieve(index, q["question"], TOP_K)
retrieval[q["id"]] = chunks
emit(f"\n{q['id']} {q['type']}: {q['question']}")
for c in chunks:
    snippet = c["text"].replace("\n", " ")
    emit(f" [{c['rank']}] score={c['score']:.4f} {c['source']} {c['chunk_id']}")
    emit(f" {snippet[:300]}{'...' if len(snippet) > 300 else ''}")
write_json(out_dir / "retrieved_top3.json", retrieval)

```

Output:

=== Retrieval (top_k=3), printed before calling the LLM ===

```

q1 [single_chunk]: In a USPL Twenty20 cricket match, how long is each session (innings), and how is that time
calculated?
[1] score=0.7586 cricket_uspl_t20_rules.txt cricket_uspl_t20_rules::0029
    bowled short. 12) Umpiring Fee for a Twenty20 Match has been fixed at $__ per umpire. USPL covers the cost.
    4 Hours of Play 1) Unless otherwise modified by the league or by the umpires on the field, the duration/
    hours of play shall be as follows; Event Timeline Coin Toss 10 minutes prior to schedu...
[2] score=0.6974 cricket_uspl_t20_rules.txt cricket_uspl_t20_rules::0039
    of the team batting 8 USPL league Rules Twenty20 - USPL Rules Committee Last Updated: 01/30/2022 second
    will be limited to the same number of overs as the team batting first (except when the team batting was
    guilty of wasting time). Also, the hours of play shall not be extended to complete the numb...
[3] score=0.6539 cricket_uspl_t20_rules.txt cricket_uspl_t20_rules::0046
    the overs shall 9 USPL league Rules Twenty20 - USPL Rules Committee Last Updated: 01/30/2022 be reduced at
    the rate of 4.25 minutes per over for the time lost. If the team batting second does not get a minimum of 5
    overs, the match shall be treated as "No Result" 8) The team batting second shall no...

q2 [two_chunks]: What is the maximum roster size for adult volleyball, and what is the time limit for a match?
[1] score=0.6687 general_douglas_county_all_sports_rules.txt general_douglas_county_all_sports_rules::0006
    for the following per sport: Flag Football - Minimum 6, Maximum 15 Soccer - Minimum 9, Maximum 20
    Softball - Minimum 10, Maximum 20 Volleyball - Minimum 6, Maximum 12 6. Players are not allowed to change
    team rosters once they have played in 1 game. 7. PENALTY: Player becomes ineligible to ...
[2] score=0.6656 volleyball_fort_collins_manual.txt volleyball_fort_collins_manual::0019
    no cap, or the one (1) hour time limit. For the third game, a coin toss will determine serve or receive and
    side. If a game is tied when time expires, the next point will decide the winner. 4. Number of Players: For
    Coed leagues, a maximum of 3 males for 6's and 2 for 4's may participate on the cou...
[3] score=0.6153 volleyball_fort_collins_manual.txt volleyball_fort_collins_manual::0018
    winner may serve/receive, or take the choice of the court for the start of play. 2. Time Limit: There will
    be a one (1) hour time limit for the match. Included in the one (1) hour time limit will be an optional five
    (5) minute warm-up period. Both teams must agree if a shorter warm-up period is dec...

q3 [similar_across_documents]: How many players per side are used in AYSO soccer for the 8U (Under-8) division?
[1] score=0.6964 soccer_ayso_national_rules.txt soccer_ayso_national_rules::0036
    no. of team members 18U 11-a-side 18 12 16U 11-a-side 18 12 14U 11-a-side 15 12 12U 9-a-side 12 10 10U 7-a-
    side 10 8 8U 4-a-side (No goalkeepers) 6 5 6U 4-a-side (No goalkeepers) 6 5 AYSO Schoolyard Soccer 4-a-side (
    No goalkeepers) 6 5 AYSO National Rules & Regulations REGISTRATION 8 AYSO National ...
[2] score=0.6480 soccer_ayso_basic_rules.txt soccer_ayso_basic_rules::0024
    number of players on the field at the time the goal was scored. This is true only if discovered by the
    referee before restarting play after the goal. Substitution In AYSO it is mandatory that all players who are
    present at the match play a minimum of one half of the game (the "Three Quarter Rule" is...
[3] score=0.6411 soccer_ayso_national_rules.txt soccer_ayso_national_rules::0025
    of the Game as m o d i f i e d b y A Y S O ( w i t h p e r m i s s i o n ) , t h e AYSO National Rules &
    Regulations and the age appropriate AYSO Coach Manual. 2. Small -sided games are permitted for all divisions.
    I. HEADING THE BALL 1. Consistent with the US Soccer mandates on heading the ball, he...

q4 [ambiguous]: What is the penalty for a foul?
[1] score=0.5938 soccer_ayso_basic_rules.txt soccer_ayso_basic_rules::0116
    Giving the coach this responsibility, which is where it belongs, is not a violation of the Laws. FIFA has
    expressly forbidden mandatory temporary expulsions, sometimes called "sit outs", by a referee for cautioned
    players to regain their composure or to 'cool off.' Serious Foul Play It is serious fo...
[2] score=0.5854 soccer_ayso_basic_rules.txt soccer_ayso_basic_rules::0016
    with a minimum of interruptions. Referees have considerable authority and flexibility when determining
    whether or not to stop play. Violation of the letter of the Law may be overlooked if the referee considers it
    to be a trifling or doubtful offense. The referee may also elect not to stop play for a...
[3] score=0.5827 soccer_ayso_basic_rules.txt soccer_ayso_basic_rules::0114
    or her own penalty area. Temporary Expulsions If a player cannot control his or her emotions and needs a '
    time -out' to regain control, either the player must recognize this and request permission to leave the field
    of play or the coach should convince the player to do so. Players have the right to ...

```

```

q5 [not_in_documents]: What is the maximum number of substitutions allowed per team in a Major League Soccer (MLS
) match?
[1] score=0.6370 basketball_las_vegas_youth_rules.txt basketball_las_vegas_youth_rules::0045
    played three. b. Any team that has more than 10 players will be allowed to substitute the eligible player(s
) half way through the quarter. The substituted player(s) must check in at the table one (1) minute prior to
check in. 2. STANDARD SUBSTITUTION PROCEDURES Player participation is controlled thr...
[2] score=0.6230 soccer_ayso_national_rules.txt soccer_ayso_national_rules::0009
    o f e a c h t e a m m a y s u b s t i t u t e a s many team members, or none, including previously
substituted team members, as long as all eligible team members meet the minimum playing requirements. e. The
referee shall allow for any time lost due to substitution o r other cause by stopping his/he...
[3] score=0.6067 general_douglas_county_all_sports_rules.txt general_douglas_county_all_sports_rules::0006
    for the following per sport: Flag Football - Minimum 6, Maximum 15 Soccer - Minimum 9, Maximum 20
Softball - Minimum 10, Maximum 20 Volleyball - Minimum 6, Maximum 12 6. Players are not allowed to change
team rosters once they have played in 1 game. 7. PENALTY: Player becomes ineligible to ...

q6 [unrelated]: What is the boiling point of water at sea level?
[1] score=0.1536 volleyball_fort_collins_manual.txt volleyball_fort_collins_manual::0015
    A Level: Designed for those teams who wish to play for competitive reasons in the highest level of
competition offered. Players in this category should have a strong volleyball background with organized
playing experience. Teams employ multiple offensive attacks and defense strategies with consisten...
[2] score=0.1421 basketball_las_vegas_youth_rules.txt basketball_las_vegas_youth_rules::0008
    8 F. Game Clock.....[long dot leader shortened]..... 9 G. Me rcy Rule .....[long dot
leader shortened].....
[3] score=0.1349 basketball_las_vegas_youth_rules.txt basketball_las_vegas_youth_rules::0043
    19 Feet, 9 inches 8 Feet 10 Feet B. Number of Players and Forfeits 1. If a team does not have at least
four players on the floor ready to play at game time, the first period will be set, 7/8/10 minutes will be
placed on the clock, and the clock will be started. Games will not start with less than f...

```

4.3 The three configurations (requirement 3)

A (no RAG): the question alone goes to the model. **B (basic RAG):** the top 3 raw chunks are pasted into the prompt, with no labels and no filtering. **C (context-engineered):** chunks scoring below 0.30 are dropped, near-duplicates (word overlap of 0.80 or more) are dropped, the survivors are ordered best-first and labelled [1], [2], ... with their source file and chunk, and the system prompt adds the grounding rules: use only the sources, cite the source number, refuse with the exact sentence “I cannot answer this question from the provided documents” when the evidence is not enough. Rules 4 and 5 (state conflicting values; do not guess on ambiguous questions) are my additions to the three the assignment lists.

Code:

```

GROUNDING_RULES = f"""You answer questions about community sports league rules using ONLY the numbered sources in
the context.
Rules:
1. Use only facts stated in the sources. Do not use outside knowledge.
2. Cite the source number in square brackets, like [1], after each fact you use.
3. If the sources do not contain enough information to answer, reply with exactly: {REFUSAL}.
4. If sources disagree, state each value and cite each source; say which file each comes from.
5. If the question is ambiguous (for example it does not say which sport), say what is ambiguous and answer only
for the interpretations the sources support, citing them. Do not guess."""

def prompt_a(question):
    return [{"role": "user", "content": question}]

def prompt_b(question, chunks):
    context = "\n\n".join(c["text"] for c in chunks)
    return [
        {"role": "system", "content": "You are a helpful assistant. Answer the question using the context
provided."},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}\nAnswer:"},
    ]

def prompt_c(question, kept):

```

```

if kept:
    context = "\n\n".join(
        f"[{i}] (source file: {c['source']}, chunk: {c['chunk_id']})\n{c['text']}"
        for i, c in enumerate(kept, 1))
else:
    context = "(No relevant sources were found.)"
return [
    {"role": "system", "content": GROUNDING_RULES},
    {"role": "user", "content": f"Sources:\n{context}\n\nQuestion: {question}\nAnswer:"},
]

def _words(text):
    return set(re.findall(r"\w+", text.lower()))

def _find_duplicate(chunk, kept, threshold):
    a = _words(chunk["text"])
    for other in kept:
        b = _words(other["text"])
        if a == b:
            return other["chunk_id"]
        if a and b and len(a & b) / len(a | b) >= threshold:
            return other["chunk_id"]
    return None

def context_engineer(chunks, min_score=MIN_SCORE, jaccard=DEDUP_JACCARD):
    """Drop chunks below 'min_score', drop near-duplicates of a higher-scoring
    chunk, and return the survivors best-first with the reason for every drop."""
    kept, dropped = [], []
    for c in sorted(chunks, key=lambda x: -x["score"]):
        if c["score"] < min_score:
            dropped.append({**c, "reason": f"score {c['score']:.3f} < min_score {min_score}"})
            continue
        dup = _find_duplicate(c, kept, jaccard)
        if dup:
            dropped.append({**c, "reason": f"near-duplicate of {dup}"})
            continue
        kept.append(c)
    return kept, dropped

def ask(client, messages):
    tin0 = getattr(client, "cumulative_input_tokens", 0)
    tout0 = getattr(client, "cumulative_output_tokens", 0)
    t0 = time.perf_counter()
    answer = client.complete(messages)
    return answer, {
        "input_tokens": getattr(client, "cumulative_input_tokens", 0) - tin0,
        "output_tokens": getattr(client, "cumulative_output_tokens", 0) - tout0,
        "latency_s": round(time.perf_counter() - t0, 2),
    }

def run_config(client, q, config, chunks, min_score):
    """Build the right prompt for A/B/C, ask the model, score the answer."""
    dropped = []
    if config == "A":
        ctx = []
        messages = prompt_a(q["question"])
    elif config == "B":
        ctx = chunks
        messages = prompt_b(q["question"], ctx)
    else:
        ctx, dropped = context_engineer(chunks, min_score)
        messages = prompt_c(q["question"], ctx)
    answer, usage = ask(client, messages)
    row = {
        "qid": q["id"], "config": config, "question": q["question"],
        "answer": answer,
        "context_chunk_ids": [c["chunk_id"] for c in ctx],
        "dropped": [{c["chunk_id": d["chunk_id"], "reason": d["reason"]} for d in dropped],
        "context_chars": sum(len(c["text"]) for c in ctx),
    }

```

```

    **usage,
    **score_row(q, config, answer, ctx),
    "messages": messages,
}
return row

```

Output (Q1 through all three configurations; A shortened):

```

[turn 1] input_tokens=36 output_tokens=3187 total_tokens=3223

--- q1 / config A (context chunks: 0, input tokens: 36) ---
In a standard Twenty20 (T20) cricket match, each session (innings) is structured as follows:

### Duration of Each Session (Innings):
- Maximum Overs: Each team bats for 20 overs (120 balls).
- Typical Time: A single innings typically lasts 1 hour to 1.5 hours, depending on the pace of play,
  fielding strategies, and interruptions (e.g., weather, delays).

### How Time is Calculated:
1. Overs and Balls:
   - Each over consists of 6 legal deliveries.
   - The total number of balls bowled in an innings is 120 (20 overs  $\times$  6 balls).
[... truncated for the report: the full answer runs about 40 more lines ...]
In summary, each session (innings) in a T20 match is 20 overs, and the time is influenced by the pace of play
, breaks, and fielding strategies, typically lasting 1-1.5 hours.

[turn 2] input_tokens=461 output_tokens=444 total_tokens=905

--- q1 / config B (context chunks: 3, input tokens: 461) ---
In a USPL Twenty20 cricket match, each session (innings) lasts 85 minutes. This time is calculated by
allocating 4.25 minutes per over for the 20 overs in an innings ( $20 \times 4.25 = 85$  minutes). The
calculation ensures consistent timing across both sessions, with no extension of play beyond the scheduled
hours unless specified by the league or umpires.

[turn 3] input_tokens=682 output_tokens=426 total_tokens=1108

--- q1 / config C (context chunks: 3, input tokens: 682) ---
In a USPL Twenty20 cricket match, each session (innings) is 85 minutes long, calculated at 4.25 minutes per
over [1]. This means each session consists of 20 overs ( $85 \div 4.25 = 20$ ). The time calculation is
standardized across both sessions, with no extension beyond this duration unless modified by the league or
umpires [1].

```

4.4 The six-question test set, including the refusals (requirement 4)

The questions and their expected behavior were committed before the first run. Q5 and Q6 must be refused, not answered.

ID	Type	Question	Expected behavior
Q1	single chunk	In a USPL Twenty20 cricket match, how long is each session (innings), and how is that time calculated?	answer
Q2	two chunks	What is the maximum roster size for adult volleyball, and what is the time limit for a match?	answer
Q3	similar across documents	How many players per side are used in AYSO soccer for the 8U (Under-8) division?	answer and flag conflict
Q4	ambiguous	What is the penalty for a foul?	clarify or answer per sport without inventing
Q5	not in documents	What is the maximum number of substitutions allowed per team in a Major League Soccer (MLS) match?	refuse
Q6	unrelated	What is the boiling point of water at sea level?	refuse

Code (loop over questions and configurations):

```
# ---- Step 3: three configurations x six questions -----
emit("\n=== Three-configuration comparison ===")
rows = []
for q in questions:
    for cfg in ("A", "B", "C"):
        row = run_config(client, q, cfg, retrieval[q["id"]], args.min_score)
        rows.append(row)
        emit(f"\n--- {q['id']} / config {cfg} "
             f"(context chunks: {len(row['context_chunk_ids'])}, input tokens: {row['input_tokens']}) ---")
        for d in row["dropped"]:
            emit(f" [dropped {d['chunk_id']}: {d['reason']}]")
        emit(row["answer"])
save_comparison(out_dir, rows)
```

Output, Q2 to Q4:

```
[turn 4] input_tokens=30 output_tokens=1178 total_tokens=1208

--- q2 / config A (context chunks: 0, input tokens: 30) ---
In adult volleyball, the standard rules are governed by the FIVB (International Volleyball Federation) and NCAA (National Collegiate Athletic Association), with slight variations depending on the league. Here's the breakdown:

Maximum Roster Size
- Standard Roster: Teams typically field 6 players on the court at a time.
- Maximum Roster: The maximum number of players a team can have on its roster is 12. This includes the 6 players on the court and additional players who may rotate in during the match (e.g., in college or professional leagues).

Match Time Limit
- No Set Time Limit: Volleyball matches do not have a fixed time limit. Instead, they are structured around sets and points:
[... truncated for the report ...]
[turn 5] input_tokens=461 output_tokens=332 total_tokens=793

--- q2 / config B (context chunks: 3, input tokens: 461) ---
The maximum roster size for adult volleyball is 12 players (as specified under the "Volleyball" entry). The time limit for a match is 1 hour, which includes an optional 5-minute warm-up period agreed upon by both teams.
[turn 6] input_tokens=684 output_tokens=710 total_tokens=1394

--- q2 / config C (context chunks: 3, input tokens: 684) ---
The maximum roster size for adult volleyball is 12 players [1]. The time limit for a match is one (1) hour, which includes an optional five (5) minute warm-up period [2].
[turn 7] input_tokens=34 output_tokens=769 total_tokens=803

--- q3 / config A (context chunks: 0, input tokens: 34) ---
In the AYSO (American Youth Soccer Organization) soccer program, the 8U (Under-8) division typically uses 7 players per side. This is part of AYSO's structured approach to youth soccer, which adjusts the number of players based on age to ensure appropriate skill development and safety.

Here's a quick breakdown of AYSO's standard player numbers by age group:
- U6 (Under 6): 5 players per side
- U8 (Under 8): 7 players per side
- U10 (Under 10): 11 players per side
- U12 and older: 11 players per side

Thus, for the 8U division, the standard number of players per side is 7.

Answer: 7 players per side.
[turn 8] input_tokens=536 output_tokens=463 total_tokens=999

--- q3 / config B (context chunks: 3, input tokens: 536) ---
The AYSO soccer 8U (Under-8) division uses 4 players per side. This is specified in the context under the "8U 4-a-side (No goalkeepers)" entry, which indicates a 4-a-side format with no goalkeepers allowed.

Answer: 4 players per side.
[turn 9] input_tokens=755 output_tokens=561 total_tokens=1316
```

```

--- q3 / config C (context chunks: 3, input tokens: 755) ---
The AYSO soccer 8U (Under-8) division uses 4 players per side [1]. This is specified in the "no. of team
members" column under the 8U entry, which lists "4-a-side (No goalkeepers)" [1]. The numbers "6 5" in the
same row likely refer to total players and substitutes, but the question specifically asks about players per
side.
[turn 10] input_tokens=18 output_tokens=766 total_tokens=784

--- q4 / config A (context chunks: 0, input tokens: 18) ---
The penalty for a foul depends on the sport, as different sports have distinct rules for fouls and their
consequences. Here are examples for some common sports:

### Soccer (Football)
- Foul: Contact with an opponent (e.g., tripping, pushing, or handling the ball).
- Penalty:
  - A free kick is awarded to the opposing team.
  - If the foul occurs in the penalty area and involves the hands (e.g., handball), a penalty kick is
    awarded.
[... truncated for the report ...]
### General Note
If you're referring to a specific sport, please clarify so I can provide a more accurate answer!
[turn 11] input_tokens=374 output_tokens=399 total_tokens=773

--- q4 / config B (context chunks: 3, input tokens: 374) ---
The penalty for a serious foul play (deliberate violence against an opponent while the ball is in play) is a
sending-off offense, which results in a red card (dismissal from the game). For less severe fouls,
the referee may stop play, award a free kick, or issue a yellow card for misconduct. Temporary expulsions ("
sit outs") are explicitly forbidden by FIFA.
[turn 12] input_tokens=589 output_tokens=708 total_tokens=1297

--- q4 / config C (context chunks: 3, input tokens: 589) ---
I cannot answer this question from the provided documents. The sources mention that serious foul play is a
sending-off offense [1], but they do not specify the penalty for all types of fouls. Additionally, the rules
about referees' discretion in handling fouls are not tied to specific penalties [2]. The documents lack
explicit details on penalties for general fouls.

```

Output, Q5 and Q6 (must be refused): configuration C refuses both with the required sentence. Configuration A answers both from memory. Configuration B says the context does not contain the answer and then answers anyway.

```

[turn 13] input_tokens=30 output_tokens=782 total_tokens=812

--- q5 / config A (context chunks: 0, input tokens: 30) ---
In Major League Soccer (MLS), the maximum number of substitutions allowed per team in a regular match is five.
This rule was introduced in 2019 to enhance tactical flexibility and game flow. Each team can make up to
five substitutions during a single match, and there is no cap on the total number of substitutions across all
teams in the league. This allows for greater strategic adaptability during games.

Answer: The maximum number of substitutions allowed per team in an MLS match is five.
[turn 14] input_tokens=440 output_tokens=2256 total_tokens=2696

--- q5 / config B (context chunks: 3, input tokens: 440) ---
The context provided does not specify the maximum number of substitutions allowed per team in a Major League
Soccer (MLS) match. The information given relates to substitution rules for various sports, including soccer,
but it primarily addresses team roster sizes (e.g., "Soccer - Minimum 9, Maximum 20") and general
substitution procedures for youth leagues (e.g., 6U, 8U, 10U divisions). For MLS, which is a professional
league, the standard substitution limit is typically 3 substitutions per team per match, but this is not
mentioned in the provided context. Therefore, based on the given text, the answer cannot be determined.

Answer: The context does not specify the maximum number of substitutions allowed per team in an MLS match.
[turn 15] input_tokens=669 output_tokens=429 total_tokens=1098

--- q5 / config C (context chunks: 3, input tokens: 669) ---
I cannot answer this question from the provided documents. The sources do not specify substitution rules for
Major League Soccer (MLS) matches. Source [2] discusses substitution rules for AYSO soccer (youth leagues),
but MLS is not mentioned in any of the provided documents.
[turn 16] input_tokens=21 output_tokens=702 total_tokens=723

```

```

--- q6 / config A (context chunks: 0, input tokens: 21) ---
The boiling point of water at sea level is 100 degrees Celsius (212 degrees Fahrenheit) under standard atmospheric pressure (1 atmosphere). This is the standard reference point for the boiling point of water in scientific contexts.

Key Notes:
- Atmospheric pressure directly affects the boiling point. At higher altitudes, where atmospheric pressure is lower, water boils at a lower temperature.
- The value 100°C is based on the standard pressure of 1 atm (approximately 101.325 kPa), which is typical at sea level.

If you're referring to a different unit or context, feel free to clarify!
[turn 17] input_tokens=303 output_tokens=274 total_tokens=577

--- q6 / config B (context chunks: 3, input tokens: 303) ---
The context provided does not contain any information about the boiling point of water. However, based on general scientific knowledge, the boiling point of water at sea level is 100°C (212°F) under standard atmospheric pressure.
[turn 18] input_tokens=181 output_tokens=136 total_tokens=317

--- q6 / config C (context chunks: 0, input tokens: 181) ---
[dropped volleyball_fort_collins_manual::0015: score 0.154 < min_score 0.3]
[dropped basketball_las_vegas_youth_rules::0008: score 0.142 < min_score 0.3]
[dropped basketball_las_vegas_youth_rules::0043: score 0.135 < min_score 0.3]
I cannot answer this question from the provided documents.

```

4.5 Sweeping top_k (requirement 5)

Q2 (needs two chunks), Q3 (conflict across documents) and Q5 (not in the documents) were each run at $k = 1, 3$ and 5 in configurations B and C. The verdicts below are the corrected ones from `rescore.py`.

Code:

```

# ---- Step 5: top_k sweep -----
emit("\n=== top_k sweep ===")
by_id = {q["id"]: q for q in questions}
sweep = []
for qid in args.sweep_questions:
    q = by_id[qid]
    for k in SWEEP_KS:
        chunks = retrieve(index, q["question"], k)
        for cfg in ("B", "C"):
            row = run_config(client, q, cfg, chunks, args.min_score)
            row.update({
                "k": k,
                "retrieved": [{"source": c["source"], "chunk_id": c["chunk_id"],
                               "score": round(c["score"], 4)} for c in chunks],
                "irrelevant_chunks_retrieved": irrelevant_count(q, chunks),
            })
            sweep.append(row)
            emit(f"{qid} k={k} cfg={cfg}: chunks_in_prompt={len(row['context_chunk_ids'])} "
                f"irrelevant_retrieved={row['irrelevant_chunks_retrieved']} "
                f"correct={yn(row['correct_answer'])} grounded={yn(row['grounded'])} "
                f"input_tokens={row['input_tokens']}")
save_sweep(out_dir, sweep)

```

Output:

```

=== top_k sweep after re-scoring (from python rescore.py) ===
q2 k=1 cfg=B: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=193
q2 k=1 cfg=C: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=358
q2 k=3 cfg=B: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=461
q2 k=3 cfg=C: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=684
q2 k=5 cfg=B: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=738
q2 k=5 cfg=C: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=1019
q3 k=1 cfg=B: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=282

```

```

q3 k=1 cfg=C: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=441
q3 k=3 cfg=B: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=536
q3 k=3 cfg=C: retrieval=NO correct=NO grounded=yes irrelevant_retrieved=0 input_tokens=755
q3 k=5 cfg=B: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=805
q3 k=5 cfg=C: retrieval=yes correct=yes grounded=yes irrelevant_retrieved=0 input_tokens=1084
q5 k=1 cfg=B: retrieval=n/a correct=NO grounded=NO irrelevant_retrieved=1 input_tokens=181
q5 k=1 cfg=C: retrieval=n/a correct=yes grounded=yes irrelevant_retrieved=1 input_tokens=342
q5 k=3 cfg=B: retrieval=n/a correct=NO grounded=NO irrelevant_retrieved=3 input_tokens=440
q5 k=3 cfg=C: retrieval=n/a correct=yes grounded=yes irrelevant_retrieved=3 input_tokens=669
q5 k=5 cfg=B: retrieval=n/a correct=NO grounded=NO irrelevant_retrieved=5 input_tokens=714
q5 k=5 cfg=C: retrieval=n/a correct=yes grounded=yes irrelevant_retrieved=5 input_tokens=1003

```

The answers behind the interesting rows (printed from `k_sweep.json`): at $k = 1$ for Q2 both configurations give the roster size and say the time limit is missing instead of guessing; at $k = 5$ for Q3 both finally see the older 5-a-side line and explain why they trust the newer table.

```

q2 k=1 config B
The maximum roster size for adult volleyball is **12 players** (as specified in the context). However, the
context does not provide information** about the time limit for a match. Therefore, the time limit cannot
be determined from the given data.
-----
q2 k=1 config C
The maximum roster size for adult volleyball is 12 [1]. The sources provided do not specify a time limit for a
match [1].
-----
q3 k=5 config B
The AYSO soccer rules specify that the **8U (Under-8) division uses 4 players per side**. This is explicitly
stated in the table under the "no. of team members" section, which lists "8U 4-a-side (No goalkeepers) 6 5."
The "4-a-side" indicates 4 players on the field per team, with no goalkeepers allowed in this division. While
the AYSO National Coaching Program recommends 5-a-side for U-8, the table reflects the standard rule for
player numbers in the 8U division.

**Answer: 4 players per side.**
-----
q3 k=5 config C
The AYSO soccer rules specify that the 8U (Under-8) division uses **4 players per side** in a 4-a-side format,
with no goalkeepers [1]. This is explicitly stated in the source. While source [4] mentions a recommendation
of 5-a-side for U-8, this is framed as a coaching guideline rather than a rule. Since the question asks about
the rules, the answer is based on the explicit rule in source [1].

Answer: 4 players per side [1].
-----

```

4.6 Evaluation (requirement 6)

For each question and configuration the code records correct retrieval (the answer-bearing text was in the context), correct answer, grounded (what the answer states is supported by the context it was given), and refused-when-needed. These are automatic heuristics, so the raw answers were also read by hand. Note that “refused when needed” counts only the exact refusal sentence: configuration B on Q5 shows NO because, after saying the context lacks the answer, it added an outside number (“typically 3 substitutions”).

Code:

```

FACT_PATTERNS = {
    "q1": [r"\b85\b", r"4\.25"],
    "q2": [r"\b12\b", r"\bhour\b\b60\s*min"],
    # Q3: match a team-size STATEMENT ("4-a-side", "4 players", "4 per side", "4v4"), not any
    # stray digit - a bare \b5\b matched the "6 5" table columns in a real answer.
    "q3": [r"\b(4|four)\W*(a\W*side|players?|per\s+side)|\b4\s*v\s*4\b",
           r"\b(5|five)\W*(a\W*side|players?|per\s+side)|\b5\s*v\s*5\b"],
}

EVIDENCE_PATTERNS = {

```

```

"q1": {"cricket_uspl_t20_rules.txt": [r"\b85\b", r".25"]},
"q2": {"general_douglas_county_all_sports_rules.txt": [r"maximum\W*12\b"],
       "volleyball_fort_collins_manual.txt": [r"\(1\)s*hour\bone hour\b1[- ]hour|hour time limit"]},
"q3": {"soccer_ayso_national_rules.txt": [r"8u\W+4\W*a\W*side"],
       "soccer_ayso_basic_rules.txt": [r"5\W*a\W*side\W+for\W+u\W*8"]},
}

def evidence_retrieved(qid, chunks):
    """True only if every expected source file contributed a chunk containing
    its answer-bearing text. None if we have no evidence patterns for qid."""
    required = EVIDENCE_PATTERNS.get(qid)
    if required is None:
        return None
    for source, patterns in required.items():
        text = " ".join(c["text"] for c in chunks if c["source"] == source).lower()
        if not all(re.search(p, text) for p in patterns):
            return False
    return True

def score_row(q, config, answer, ctx_chunks):
    qid, behavior = q["id"], q["expected_behavior"]
    refused = is_refusal(answer)
    a = _strip_cites(answer).lower()
    ctx_text = _strip_cites("\n".join(c["text"] for c in ctx_chunks)).lower()
    ctx_sources = sorted({c["source"] for c in ctx_chunks})
    facts = FACT_PATTERNS.get(qid, [])

    # -- correct answer -----
    if behavior == "refuse":
        correct = refused
    elif behavior == "answer":
        correct = (not refused) and all(re.search(p, a) for p in facts)
    elif behavior == "answer_and_flag_conflict":
        correct = (not refused) and all(re.search(p, a) for p in facts)
    else: # clarify_or_answer_per_sport_without_inventing
        correct = (not refused) and (bool(re.search(CLARIFY_CUES, a)) or len(_sports_in(a)) >= 2)

    # -- correct retrieval (only meaningful for RAG configs with expected sources)
    if config == "A" or not q["expected_sources"]:
        correct_retrieval = None
    else:
        correct_retrieval = evidence_retrieved(qid, ctx_chunks)

    # -- grounded: is what the answer states supported by the context it was given
    if config == "A":
        grounded = None
    elif refused:
        grounded = True
    elif behavior == "refuse":
        grounded = False # it answered something the documents cannot support
    elif facts:
        in_answer = [p for p in facts if re.search(p, a)]
        grounded = bool(in_answer) and all(re.search(p, ctx_text) for p in in_answer)
    else:
        grounded = _sports_in(a) <= _sports_in(ctx_text)

    cites = bool(re.search(r"\[d+\]", answer))
    refusal_needed = behavior == "refuse"
    return {
        "correct_retrieval": correct_retrieval,
        "correct_answer": bool(correct),
        "grounded": grounded,
        "refused": refused,
        "refused_when_needed": refused if refusal_needed else None,
        "false_refusal": refused and not refusal_needed,
        "cites_source": cites,
        "format_ok": (refused or cites) if config == "C" else None,
        "context_sources": ctx_sources,
    }

```

```

def _rate(values):
    vals = [v for v in values if v is not None]
    return None if not vals else round(sum(1 for v in vals if v) / len(vals), 3)

def summarize(rows):
    out = {}
    for cfg in ("A", "B", "C"):
        r = [x for x in rows if x["config"] == cfg]
        out[cfg] = {
            "accuracy": _rate([x["correct_answer"] for x in r]),
            "faithfulness": _rate([x["grounded"] for x in r]),
            "format_compliance": _rate([x["format_ok"] for x in r]),
            "robustness": _rate([x["correct_answer"] for x in r if x["qid"] in STRESS_QUESTIONS]),
        }
    return out

```

Per-question results (corrected):

Q	Config	Correct retrieval	Correct answer	Grounded	Refused when needed	Cites source
Q1	A	n/a	NO	n/a	n/a	NO
Q1	B	yes	yes	yes	n/a	NO
Q1	C	yes	yes	yes	n/a	yes
Q2	A	n/a	NO	n/a	n/a	NO
Q2	B	yes	yes	yes	n/a	NO
Q2	C	yes	yes	yes	n/a	yes
Q3	A	n/a	NO	n/a	n/a	NO
Q3	B	NO	NO	yes	n/a	NO
Q3	C	NO	NO	yes	n/a	yes
Q4	A	n/a	yes	n/a	n/a	NO
Q4	B	n/a	NO	yes	n/a	NO
Q4	C	n/a	NO	yes	n/a	yes
Q5	A	n/a	NO	n/a	NO	NO
Q5	B	n/a	NO	NO	NO	NO
Q5	C	n/a	yes	yes	yes	yes
Q6	A	n/a	NO	n/a	NO	NO
Q6	B	n/a	NO	NO	NO	NO
Q6	C	n/a	yes	yes	yes	NO

Summary (corrected):

Configuration	Accuracy	Faithfulness	Format compliance	Robustness (Q3–Q6)
A: No RAG	17%	n/a	n/a	25%
B: Basic RAG	33%	67%	n/a	0%
C: Context-engineered	67%	100%	100%	50%

Accuracy is the share of the six questions handled as expected (right answer, or a refusal where one was needed). Faithfulness is the share of RAG answers supported by their context. Format compliance is the share of configuration C answers that cite a [n] source or use the exact refusal sentence (A and B were never told to, so n/a). Robustness is accuracy on the four stress questions, Q3 to Q6.

A scoring bug found and fixed. The first scoring pass marked Q3 in configuration C as correct because a stray “5” in a table column (“6 5”) matched the check for the older 5-a-side statement. Reading the raw answers showed the model had not mentioned the conflict at all. The check was tightened and every saved answer was re-scored with `rescore.py` (no new model calls). Only Q3 in

configuration C changed, in the main table and in the $k = 3$ sweep row. The original files are kept in `reports/hw04/raw/first_pass_scoring/`.

Config C metric	First scoring pass	After correction
Accuracy	83%	67%
Robustness (Q3–Q6)	75%	50%
Faithfulness / format compliance	100% / 100%	100% / 100%

4.7 Written analysis (requirement 7)

Which questions found the right chunks

Q1 and Q2 found what they needed in the top three chunks. Q1’s top chunk held both “85 minutes” and “4.25 minutes per over”. Q2 found the roster limit in one document and the one-hour limit in another. Q3 failed: the top three chunks had the 2018 table (4-a-side) but not the older booklet that says 5-a-side, so the model could not see the conflict. Q4 (ambiguous) returned three soccer chunks, so the “which sport?” problem was lost before the model saw anything. Q5’s wrong-but-similar chunks scored 0.61 to 0.64, almost as high as the real answers to Q2 (0.62 to 0.67), so a score cutoff could not catch them. Q6’s chunks scored only 0.13 to 0.15.

Which changes helped

The biggest help was the grounding rules and source labels in config C, not the filtering. The score filter only mattered on Q6: it removed all three chunks, and the model then refused. Removing duplicates did nothing here. The labels let the model cite sources, and the rules made it refuse when the evidence was missing. Accuracy was 17% with no RAG, 33% with basic RAG and 67% with context-engineered RAG. Basic RAG only got the two easy questions right. Context-engineered RAG was also fully faithful (100%, against 67% for basic RAG) and always followed the required format.

Did the model make things up on Q5 and Q6?

Yes, in two of the three setups. With no RAG, it answered both from memory. Basic RAG was trickier: it said the documents did not have the answer, then gave one anyway (100 degrees C for Q6, and “typically 3” substitutions for Q5, while the no-RAG run said five). Those two numbers disagree, so neither can be trusted. Context-engineered RAG refused both with the exact required sentence.

What the k sweep showed

More context was not always better. Q2 needed $k=3$. At $k=1$, both setups gave the roster size and honestly said the time limit was missing. Going to $k=5$ cost about 280 to 335 more tokens and added nothing. Q3 was the opposite: it only worked at $k=5$, when the older 5-a-side line finally appeared. Both setups then answered 4 per side and explained why they trusted the newer table. On Q5, context-engineered RAG refused at $k=1, 3$ and 5 , while basic RAG made things up every time. So the best k depends on the question.

Limits

Only one chunk size (500 characters) was tested. The scores come from automatic checks. One scoring bug (Q3, config C) was found by reading the answers and fixed, and both the original and corrected tables are saved. Also, temperature 0 did not give identical runs: repeated prompts gave different answer lengths, though the verdicts stayed the same.